

SQL 과목 III 튜닝 스타일 모의문제 · 11회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

인덱스를 경유해 테이블 행을 읽는 처리의 비용이 어디에서 발생하는지, 그리고 클러스터링 팩터가 그 비용의 어느 부분에 작용하는지에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스를 경유한 테이블 액세스에서 실제 비용의 상당 부분은 리프에서 얻은 ROWID로 테이블 블록을 한 건씩 찾아가는 랜덤 액세스(Single Block I/O)에서 나오며, 조건을 만족하는 행이 많아질수록 이 테이블 랜덤 액세스 횟수가 늘어 부담이 커진다.
- ② 클러스터링 팩터는 인덱스 리프 엔트리들이 리프 블록 안에 얼마나 촘촘히 정렬돼 담겼는지를 나타내는 값이라 인덱스 수직 탐색과 리프 스캔 비용을 좌우할 뿐, 리프의 ROWID로 테이블 블록을 찾아가는 랜덤 액세스의 블록 방문 횟수와는 관계가 없다.
- ③ 같은 건수의 행을 읽더라도 클러스터링 팩터가 좋아 인접한 ROWID가 같은 테이블 블록에 모여 있으면, 한 번 읽어 캐시에 올린 블록에서 여러 행을 이어 처리해 실제 Single Block I/O로 방문하는 테이블 블록 수가 줄어든다.
- ④ 인덱스 자체의 정렬 상태가 아무리 좋아도 테이블의 저장 순서가 인덱스 순서와 어긋나 클러스터링 팩터가 나쁘면, 넓은 범위를 인덱스로 읽을 때 행마다 다른 테이블 블록을 랜덤 액세스하게 되어 오히려 Full Table Scan보다 불리해질 수 있다.

2

B-Tree 인덱스가 하나의 세그먼트로서 디스크에 저장되는 구조 — 루트·브랜치·리프 블록의 구성과 인덱스 높이 — 에 대한 설명으로 가장 적절한 것은?

- ① B-Tree 인덱스는 그 자체로 하나의 세그먼트이며 루트·브랜치·리프 블록으로 이뤄진다. 조회는 루트에서 브랜치를 거쳐 리프에 이르는 수직 탐색으로 시작 리프를 찾고, 인덱스 높이(루트에서 리프까지의 레벨 수)는 대체로 낮게 유지되며 특정 블록이 가득 차면 블록 분할(split)로 균형을 잡아 준다.
- ② 'B-Tree'의 B가 이진(Binary)을 뜻하므로 루트 블록은 자식을 정확히 둘 가리키고 그 아래 브랜치 블록도 저마다 자식을 둘씩만 거느린 이진 트리를 이룬다. 따라서 루트에서 리프에 이르는 수직 탐색의 레벨 수, 곧 인덱스 높이는 전체 리프 블록 수를 밑이 2인 로그로 취한 값에 비례해 정해지고, 리프가 늘수록 높이도 그만큼 가파르게 깊어진다.
- ③ 인덱스 세그먼트에 디스크로 담기는 것은 키와 ROWID를 실은 리프 블록뿐이고, 루트와 브랜치는 세그먼트 밖의 별도 메모리 구조에만 유지된다. 그래서 루트에서 브랜치를 거치는 수직 탐색 구간은 메모리 안에서 끝나고, 블록이 가득 차 일어나는 분할(split) 역시 리프에서만 생기므로 인덱스가 커져도 세그먼트에 쌓이는 것은 리프 블록뿐이다.
- ④ 일반 B-Tree 인덱스는 테이블과 합쳐져 하나의 세그먼트를 이루고, 그 리프 블록에는 인덱스 키와 ROWID가 아니라 행 데이터 전체가 함께 담긴다. 그래서 루트에서 브랜치를 거쳐 리프에 닿는 수직 탐색이 끝나는 순간, 테이블을 따로 되짚지 않고도 인덱스에 없는 컬럼까지 그 자리에서 값을 얻어 낼 수 있다.

국가기록물관리원의 전자기록물 색인 배치가 이관된 문서 500,000건을 순회하며 건마다 분류코드 갱신 UPDATE 1건씩(총 500,000건)을 실행한다. 개발자가 EXECUTE IMMEDIATE로 SQL 문자열을 조립하면서 기록물번호·분류코드 같은 값을 바인드 변수가 아니라 리터럴 문자열로 이어 붙여 동적 SQL을 만들었기 때문에 실행마다 SQL 텍스트가 조금씩 달라진다. 6개의 색인 세션이 동시에 돌아 응답이 50초로 늘고 대기 이벤트 상위에 library cache: mutex X·cursor: pin S wait on X·shared pool latch가 잡혔다. 아래는 이 배치 세션의 비재귀·재귀 statement를 나눠 집계한 TKPROF이다. 지연의 원인을 직접 지목한 것으로 옳은 것은? (단, 옵티마이저 통계는 최신이고 Shared Pool은 여유가 충분해 에이징으로 인한 강제 재파싱은 없으며, 각 실행은 조건값만 다르고 UPDATE 구조는 동일하다.)

아 래							
OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	500000	40.000	42.000	0	0	0	0
Execute	500000	6.000	8.000	0	1000000	500000	500000
Fetch	0	0.000	0.000	0	0	0	0
Total	1000000	46.000	50.000	0	1000000	500000	500000
Misses in library cache during parse: 485000							
OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	1000000	7.000	7.000	0	0	0	0
Execute	1000000	9.000	9.000	0	8000000	0	0
Fetch	3000000	5.000	5.000	0	12000000	0	3000000
Total	5000000	21.000	21.000	0	20000000	0	3000000
Misses in library cache during parse: 25							

- ① 재귀 statement가 Execute에서 Query 800만, Fetch에서 1,200만을 합쳐 논리 읽기 2,000만을 쌓았고 재귀 Total CPU도 21.0초에 이른다. 데이터 디크셔너리 블록을 과도하게 읽어 대는 이 재귀 측이 병목이므로, DB 버퍼 캐시를 키워 디크셔너리 블록을 캐시에 상주시켜 재귀 논리 읽기를 건너 내면 파싱 지연이 사라진다.
- ② 비재귀 Parse 500,000콜과 Execute 500,000콜이 1:1인데 그 Parse에만 CPU 40.0초·Elapsed 42.0초가 몰려 있어, 커서를 붙잡지 못해 실행마다 파싱을 되풀이하는 소프트파싱 폭주가 병목이다. session_cached_cursors를 늘리고 커서를 홀드해 Parse 콜을 없애면 library cache: mutex X 대기과 50초가 함께 줄어든다.
- ③ 라이브러리 캐시 미스가 485,000건에 달해 사실상 매 실행이 하드파싱이고, 그 하드파싱이 데이터 디크셔너리를 뒤지며 재귀 Call 500만을 유발한 것이 원인이다. 동적 SQL의 리터럴을 바인드 변수로 바꿔 SQL 텍스트를 하나로 고정하면 하드파싱이 소프트파싱으로 바뀌어 50초가 해소된다.
- ④ 비재귀 Execute가 500,000콜로 Rows 500,000을 갱신하며 Query 1,000,000·Current 500,000 논리 읽기와 CPU 6.0초·Elapsed 8.0초를 썼다. 갱신 1건마다 Query 2블록·Current 1블록을 만지는 이 실행 측이 작업의 골격이므로, UPDATE 실행계획을 다시 짜 블록 접근을 줄이면 50초가 해소된다.

EXPLAIN PLAN으로 적재한 예상 실행계획을 DBMS_XPLAN.DISPLAY로 조회했을 때, 출력된 계획 트리를 어떻게 읽어야 하는지 — 단계의 실행 순서, Predicate 섹션의 두 조건, format 수준 — 에 대한 설명으로 가장 적절한 것은?

- ① EXPLAIN PLAN으로 적재한 계획을 DBMS_XPLAN.DISPLAY로 뽑으면 Id가 0부터 차례로 매겨져 나오므로, 단계의 실행 순서도 표에 적힌 위에서 아래 순서 그대로다. Id가 앞선 Operation이 항상 먼저 수행되고, 들여쓰기 깊이는 부모-자식 관계를 눈에 띄게 하려고 넣은 표시상의 장식일 뿐이어서 실행 순서와는 무관하다.
- ② 계획 하단 Predicate 섹션은 WHERE 절 조건을 어느 단계에 걸린 것인지 번호와 함께 나열한 것인데, 거기 적히는 access 조건과 filter 조건은 붙은 이름만 다를 뿐 하는 일이 같다. 그래서 어느 쪽에 적혀 있든 그 조건이 해당 단계에서 인덱스나 조인의 탐색 범위를 좁혀 준다는 뜻으로 읽으면 되고, 두 이름을 갈라 볼 필요는 없다.
- ③ DBMS_XPLAN.DISPLAY의 format 인자를 기본값 'TYPICAL'에서 'BASIC'으로 낮추면 Operation·Name에 더해 Cost·카디널리티는 물론 Predicate 섹션의 access·filter 조건까지 가장 상세히 함께 출력된다. 그러므로 각 단계가 어떤 조건으로 진입하고 무엇을 걸러 내는지 한눈에 보려면 'ALL'이 아니라 'BASIC'을 주면 된다.
- ④ 계획 트리에서 단계의 실행 순서는 적힌 차례가 아니라 들여쓰기가 드러내는 부모-자식 관계로 읽어야 하며, 가장 깊은 자식부터 수행해 결과를 부모로 올린다. Predicate 섹션의 access 조건은 진입 시 탐색 범위를 좁히는 조건이고, filter 조건은 그렇게 얻은 행을 추가로 걸러 내는 조건이라 둘을 구분해 읽어야 한다.

재외국민 영사업무 시스템에서 재외국민등록부(약 300건)를 드라이빙해 영사업무접수(약 3억 건)를 NL 조인으로 1,500,000행 뽑는 SQL의 TKPROF이다. 개발자는 물리 읽기(Disk)가 작으니 캐시 적중이 높아 빠를 것으로 봤는데, 응답이 45초를 넘겨 느리다는 신고가 들어왔다. 아래 트레이스에 대한 해석으로 가장 적절한 것은? (단, 옵티마이저 통계는 최신이고 CPU 자원의 외부 경합은 없으며, 영사업무접수의 인덱스 영사업무접수_N1을 경유하는 NL 조인으로만 수행되고 조인 대상 블록은 대부분 이미 버퍼 캐시에 올라와 있어 물리 I/O가 작다.)

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.020	0.020	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	1501	36.000	45.000	180000	3000000	0	1500000
Total	1503	36.020	45.020	180000	3000000	0	1500000
Rows	Row Source Operation						
1500000	NESTED LOOPS (cr=3000000 pr=180000 pw=0 time=44000000 us)						
300	TABLE ACCESS FULL 재외국민등록부 (cr=30 pr=0 pw=0 time=800 us)						
1500000	TABLE ACCESS BY INDEX ROWID 영사업무접수 (cr=2999970 pr=180000 pw=0 time=36000000 us)						
1500000	INDEX RANGE SCAN 영사업무접수_N1 (cr=1900000 pr=0 pw=0 time=11000000 us)						

- ① 물리 읽기 pr=180,000이 영사업무접수 노드 한 곳에 통째로 몰려 있고 그 노드의 time도 36,000,000us로 NL 조인 전체 44초의 대부분을 차지한다. 디스크에서 블록을 실어 올리는 이 180,000번의 단일 블록 읽기가 45초의 뿌리이므로, DB 버퍼 캐시를 키워 해당 블록을 상주시켜 물리 읽기를 없애면 응답이 최적화된다.
- ② Fetch의 Elapsed 45.000에서 CPU 36.000을 뺀 9.0초가 고스란히 물리 읽기 대기이고, 그 대기는 Disk 180,000 블록을 실어 올리느라 생긴 것이다. 영사업무접수 노드에 pr 180,000이 몰린 것까지 보면 이 SQL은 I/O 바운드이므로, 병렬도(DOP)를 높여 디스크 읽기를 여러 프로세스로 나눠 주면 45초가 개선된다.
- ③ BCHR이 94.0%로 높아 물리 읽기(Disk 180,000)는 논리 읽기의 6%뿐인데도 CPU가 45초의 80%를 차지한다. 병목은 디스크가 아니라 NL 조인이 영사업무접수를 150만 번 방문하며 쌓은 논리 읽기 300만이 CPU를 태우는 것이므로, 조인 방식·액세스 경로를 바꿔 논리 읽기 횟수 자체를 줄여야 한다.
- ④ NESTED LOOPS 노드가 1,500,000행을 내놓으며 time=44,000,000us를 쓴 것은, 이 결과 집합이 해시·정렬 작업 영역을 넘겨 temp로 흘렀기 때문이다. 드라이빙 재외국민등록부 300건에 건줘 결과가 150만 행으로 불어난 만큼 작업 영역이 모자란 것이므로, PGA를 키워 이 spill을 없애면 45초가 사라진다.

국립수목원 식물표본 관리 시스템의 표본관찰기록 테이블(약 880만 건)에는 결합 인덱스 관찰_IX = (수목원코드, 관찰일자)가 있다. 관찰이 이뤄진 수목원명을 함께 얻기 위해 수목원 테이블(기본키 수목원_PK = 수목원코드)과 조인하는 다음 SQL의 실행계획이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아래

```
SELECT s.표본번호, s.관찰일자, s.표본명, a.수목원명
FROM 표본관찰기록 s, 수목원 a
WHERE s.수목원코드 = 'K12'
      AND s.관찰일자 >= DATE '2026-05-01'
      AND s.관찰일자 < DATE '2026-06-01'
      AND s.표본상태 = '고사'
      AND a.수목원코드 = s.수목원코드;
```

Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(362	1650	102300)
1	0	NESTED LOOPS	(362	1650	102300)
2	1	TABLE ACCESS BY INDEX ROWID 표본관찰기록	(355	1650	69300)
3	2	INDEX RANGE SCAN 관찰_IX	(21	6300	)
4	1	TABLE ACCESS BY INDEX ROWID 수목원	(1	1	20)
5	4	INDEX UNIQUE SCAN 수목원_PK	(0	1	)

- ① 수목원코드(등치)와 관찰일자(범위)까지가 관찰_IX의 인덱스 액세스 조건으로 INDEX RANGE SCAN에서 처리(Card 6,300)되고, 인덱스에 없는 표본상태는 TABLE ACCESS BY INDEX ROWID 단계 필터로 걸려져 Card가 1,650으로 줄어든다.
- ② WHERE의 세 조건(수목원코드 등치·관찰일자 범위·표본상태 등치)이 모두 관찰_IX의 인덱스 액세스 조건으로 Id 3 INDEX RANGE SCAN 단계에서 함께 처리되어 스캔 범위를 결정하며, 그 덕분에 이 단계에서 이미 Card 6,300까지 좁혀진다.
- ③ 관찰_IX의 두 번째 칼럼인 관찰일자가 범위(>=, <) 조건이므로 선두 칼럼 수목원코드의 등치까지 액세스 조건 자격을 잃어, Id 3 INDEX RANGE SCAN은 인덱스 리프 전 구간을 훑은 뒤 두 조건을 스캔 후 필터로만 걸러 Card 6,300을 내놓는다.
- ④ Id 3 INDEX RANGE SCAN이 반환한 Card 6,300이 곧 이 SQL의 최종 결과 건수이며, Id 2 TABLE ACCESS BY INDEX ROWID는 그 6,300건의 ROWID로 표본명처럼 인덱스에 없는 컬럼을 채워 넣을 뿐 건수 감소를 일으키지 않는다.

풍력발전 통합관제 시스템에서 터빈출력계측(약 8,000만 건, 테이블 총 블록 약 80,000)을 단지코드 인덱스로 검색해 특정 단지의 계측 300,000행을 인덱스 순서로 읽어 내려받는 SQL의 TKPROF이다. 인덱스로 잘 타는데도 응답이 50초를 넘긴다. 개발자는 "Fetch가 여러 번이라 인출 왕복이 병목"이라 보고 배열 크기를 더 키우려 하고, 옆자리 동료는 "물리 읽기가 문제"라며 버퍼 캐시를 키우자고 한다. 아래 트레이스를 배열 크기(Array Processing)·클러스터링 팩터·BCHR 세 지표로 함께 진단할 때, 실제 병목과 처방을 옳게 종합한 것은? (단, 옵티마이저 통계는 최신이고, 애플리케이션은 이미 배열 인출을 사용하며 인덱스는 단지코드 단일 컬럼으로 구성돼 있다.)

아래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.003	0.003	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	401	8.000	50.000	30000	300000	0	300000
Total	403	8.003	50.003	30000	300000	0	300000
Rows	Row Source Operation						
300000	TABLE ACCESS BY INDEX ROWID 터빈출력계측 (cr=300000 pr=30000 pw=0 time=49000000 us)						
300000	INDEX RANGE SCAN 터빈출력계측_단지_IDX (cr=3000 pr=100 pw=0 time=800000 us)						

- ① Fetch가 401회나 발생했고 그 Fetch에 Elapsed 50.000초가 통째로 잡혀 있다. 배열 크기가 작아 300,000행을 401번에 나누어 나르는 인출 왕복이 50초를 만든 것이며, CPU가 8.000초뿐인 것도 시간 대부분이 서버-클라이언트 왕복 대기로 흘렀다는 증거다. 배열 크기를 더 키워 Fetch 횟수를 줄이면 왕복이 사라져 해소된다.
- ② 물리 읽기 pr 30,000이 테이블 노드에 물려 있고 그 노드 time이 49,000,000us로 전체 50초의 대부분을 차지한다. CPU는 8.000초뿐이라 나머지는 디스크 대기이고 BCHR도 90.0%에 그쳐 물리 10%가 남아 있으므로, DB 버퍼 캐시를 키워 이 30,000 블록을 캐시 적중으로 돌리면 랜덤 액세스 대기가 사라진다.
- ③ 배열 크기는 750으로 이미 커 왕복은 병목이 아니고, BCHR 90.0%라 물리 읽기는 10%뿐이다. 반면 클러스터링 팩터 밀도가 0.99로 최댓값 1에 근접해, 인덱스로 뽑은 행마다 흩어진 블록을 랜덤 액세스한 것이 병목이다. 인덱스 키 순서로 테이블을 재정렬하거나 커버링 인덱스로 테이블 액세스를 없애야 한다.
- ④ 테이블 노드 cr 300,000은 Rows 300,000과 1:1이라 행 하나마다 블록을 한 번씩 읽은 셈이고, 이 논리 읽기 30만이 CPU 8.000초와 노드 time 49초를 만든 병목이다. 인덱스 터빈출력계측_단지_IDX를 리빌드해 리프를 촘촘히 다지고 단편화를 걷어 내 인덱스 스캔의 논리 읽기(cr 3,000)를 줄이면 50초가 개선된다.

인덱스 전체를 읽는 두 방식인 Index Full Scan과 Index Fast Full Scan이 각각 어떤 방식으로 블록을 읽는지, 그리고 인덱스 전용 스캔(커버링)과 어떻게 맞물리는지에 대한 설명으로 가장 적절하지 않은 것은?

- ① Index Full Scan은 인덱스 리프 블록을 논리적 연결 순서가 아니라 물리적 저장 순서대로 한꺼번에 Multiblock I/O로 읽으므로, 대량 데이터를 훑을 때 Index Fast Full Scan보다 빠르고 병렬 처리에도 더 잘 맞는다.
- ② Index Full Scan은 리프 블록을 좌우 논리적 연결 순서를 따라 한 블록씩 읽어 인덱스 키 순서로 정렬된 결과를 내므로, 정렬이 필요한 조회에서 ORDER BY를 위한 별도 소트를 생략하는 데 쓸 수 있다.
- ③ Index Fast Full Scan은 인덱스 세그먼트의 블록을 물리적 저장 순서대로 Multiblock I/O로 읽어 대량 처리와 병렬 수행에 유리한 대신, 리프의 연결 순서를 따르지 않아 결과가 인덱스 키 순서로 정렬되어 나오지는 않는다.
- ④ 인덱스 전용 스캔(커버링)은 스캔 방식과는 별개의 성질이라, 쿼리가 참조하는 컬럼이 인덱스에 모두 들어 있으면 Index Full Scan이든 Index Fast Full Scan이든 그 방식과 무관하게 테이블(TABLE ACCESS BY INDEX ROWID)을 되짚지 않고 인덱스만 읽어 결과를 낸다.

9

결합 인덱스의 컬럼 순서를 정하고, 리프가 곧 데이터인 클러스터형 인덱스(SQL Server)·IOT(오라클)를 어느 테이블에 둘지 판단하는 인덱스 설계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 결합 인덱스는 변별력이 큰(카디널리티가 높은) 컬럼을 선두에 두는 것이 최적이므로, 조회에서 그 컬럼이 어떤 형태의 조건으로 쓰이는지나 조건절에 등장하는지와 무관하게, 카디널리티가 가장 높은 컬럼을 맨 앞에 배치하기만 하면 어느 쿼리에서든 스캔 범위가 가장 좁아진다.
- ② 결합 인덱스의 컬럼 순서는 조회 패턴을 따라야 하며, 선두 컬럼은 그 조회에서 등가(=) 조건으로 자주 쓰이는 컬럼이라야 스캔 시작·종료 지점을 좁게 잡을 수 있다. 조건절에 아예 등장하지 않는 컬럼은 앞에 두어도 그 조회의 액세스 범위를 좁혀 주지 못한다.
- ③ 클러스터형 인덱스(IOT)는 리프 레벨이 곧 정렬된 행 데이터라 테이블의 물리적 저장 순서를 결정하므로 한 테이블에 하나만 둘 수 있어, 그 테이블에서 가장 이득이 큰 액세스 경로(범위 검색이 잦은 키)를 골라 신중히 배정해야 한다.
- ④ 인덱스 설계는 조회 패턴만이 아니라 컬럼 추가·순서 변경이 부르는 인덱스 크기 증가와 DML 유지 부하를 함께 견주어야 하며, 얻는 조회 이득과 그 유지 비용 사이에는 맞교환되는 손익분기점이 있다.

10

수산물 위판 정산 조회 프로그램이 위판장(약 220건)·위판낙찰(약 190만 건)·낙찰명세(약 1억 2천만 건) 세 테이블을 조인해 특정 위판장의 낙찰 명세를 뽑는다. 위판낙찰는 위판장코드로 위판장과, 낙찰명세는 낙찰번호로 위판낙찰와 이어진다(위판장과 낙찰명세를 직접 잇는 조인 칼럼은 없다). 네 후보 SQL은 SELECT 목록·WHERE 조건이 모두 같고 조인 힌트만 서로 다르다. 아래 실행계획을 낸 SQL/힌트로 가장 적절하지 않은 것은?

아래

Id	Operation	Name
0	SELECT STATEMENT	
1	NESTED LOOPS	
2	NESTED LOOPS	
* 3	TABLE ACCESS FULL	위판장
4	TABLE ACCESS BY INDEX ROWID	위판낙찰
* 5	INDEX RANGE SCAN	위판낙찰_IX
6	TABLE ACCESS BY INDEX ROWID	낙찰명세
* 7	INDEX RANGE SCAN	낙찰명세_IX

① 아래 SQL

```
SELECT /*+ LEADING(g e d) USE_NL(e) USE_NL(d) */
      g.위판장명, e.낙찰일자, d.어종코드, d.낙찰단가
FROM 위판장 g, 위판낙찰 e, 낙찰명세 d
WHERE e.위판장코드 = g.위판장코드
      AND d.낙찰번호 = e.낙찰번호
      AND g.위판구분 = '수산';
```

② 아래 SQL

```
SELECT /*+ LEADING(e g d) USE_NL(g) USE_NL(d) */
      g.위판장명, e.낙찰일자, d.어종코드, d.낙찰단가
FROM 위판장 g, 위판낙찰 e, 낙찰명세 d
WHERE e.위판장코드 = g.위판장코드
      AND d.낙찰번호 = e.낙찰번호
      AND g.위판구분 = '수산';
```

③ 아래 SQL

```
SELECT /*+ ORDERED USE_NL(e) USE_NL(d) */
      g.위판장명, e.낙찰일자, d.어종코드, d.낙찰단가
FROM 위판장 g, 위판낙찰 e, 낙찰명세 d
WHERE e.위판장코드 = g.위판장코드
      AND d.낙찰번호 = e.낙찰번호
      AND g.위판구분 = '수산';
```

④ 아래 SQL

```
SELECT /*+ LEADING(g e d) USE_NL(e d) */
      g.위판장명, e.낙찰일자, d.어종코드, d.낙찰단가
FROM 위판장 g, 위판낙찰 e, 낙찰명세 d
WHERE e.위판장코드 = g.위판장코드
      AND d.낙찰번호 = e.낙찰번호
      AND g.위판구분 = '수산';
```

11

서브쿼리를 담은 조건(IN/EXISTS, NOT IN/NOT EXISTS)과 집합 연산자 MINUS가 각각 세미(semi) 조인·안티(anti) 조인과 맺는 관계에 대한 설명으로 가장 적절한 것은?

- ① IN 서브쿼리는 '짜이 하나라도 있으면 통과'라는 세미 조인의 대표 사례이므로, 서브쿼리 쪽 컬럼이 유일키여서 왼쪽 한 행에 두 건 이상이 매칭될 수 없는 경우에도 옵티마이저는 이를 일반 조인으로 풀지 않고 HASH JOIN SEMI 같은 세미 조인으로만 변환한다. IN이라는 키워드 자체가 세미 조인 변환의 조건이기 때문이다.
- ② EXISTS와 IN은 '짜이 하나라도 있으면 통과'라는 의미여서 세미 조인으로, NOT EXISTS와 NOT IN은 '짜이 하나도 없어야 통과'라는 의미여서 안티 조인으로 풀리며, MINUS(차집합)도 앞 집합에서 뒤 집합에 없는 행만 남긴다는 점에서 '짜 없는 행만 통과'하는 안티 조인과 성격이 통한다.
- ③ MINUS는 '뒤 집합에 짜이 없는 행만 남긴다'는 점에서 안티 조인과 완전히 같은 연산이라, 옵티마이저는 MINUS 쿼리를 그대로 HASH JOIN ANTI로 바꿔 실행한다. NOT EXISTS가 안티로 풀릴 때와 결과 집합도 같으므로, 앞뒤 집합의 중복을 걷어 내기 위한 정렬·해시 단계는 따로 생기지 않는다.
- ④ EXISTS·NOT EXISTS의 상관 조건이 비등치(>= 같은 범위 비교)이면 해시 테이블의 키로 매칭할 수 없어 HASH JOIN SEMI/ANTI가 만들어지지 않는다. 세미·안티는 해시 매칭을 전제로 하는 변환이므로, 이런 경우 옵티마이저는 변환을 포기하고 서브쿼리를 바깥 행마다 다시 수행하는 단순 필터로만 처리한다.

12

옵티마이저의 뷰 병합(View Merging) — 단순 뷰의 병합, 복합 뷰(집계·중복 제거 포함)의 병합, 병합이 불가능한 경우, 그리고 NO_MERGE/MERGE 힌트 — 에 대한 설명으로 가장 적절하지 않은 것은?

- ① 단순 뷰(집계·중복 제거가 없는 뷰)의 병합은 뷰 정의를 바깥 쿼리 블록에 그대로 풀어 넣어 하나의 블록으로 합치는 변환으로, 뷰 경계가 사라져 옵티마이저가 조인 순서와 액세스 경로를 더 넓게 탐색할 수 있다.
- ② GROUP BY·DISTINCT가 든 복합 뷰도 조건이 맞으면 병합될 수 있으며(복합 뷰 Merging), 이때는 집계·중복 제거 연산의 수행 시점까지 함께 옮겨 재구성한다.
- ③ 뷰 안에 ROWNUM이나 집합 연산자(UNION 등), 또는 분석함수처럼 병합하면 의미가 달라지는 요소가 들어 있으면 그 뷰는 병합되지 못하고, 하나의 독립된 블록으로 수행된 뒤 바깥 쿼리와 결합된다.
- ④ 옵티마이저가 병합할 수 없다고 판단한 뷰라도 MERGE 힌트를 주면 병합이 성사되므로, ROWNUM이나 UNION이 든 뷰도 힌트만 붙이면 바깥 쿼리와 한 블록으로 합쳐진다.

13

반려동물 등록 관리 시스템의 조회 모듈이 동물등록원부(약 1,900만 건, 등록번호 유일)에서 동물 한 마리를 등록번호로 찾는 단건 조회를 초당 수백 회 발행한다. 초기 버전은 등록번호를 리터럴로 문자열에 이어 붙여 SQL을 만들었고(아래 위쪽), 파싱 부하가 커지자 바인드 변수로 고쳐 발행하도록 바꿨다(아래 아래쪽). 두 방식의 커서 공유·라이브러리 캐시 동작에 대한 설명으로 가장 적절하지 않은 것은? (단, 오라클이며 CURSOR_SHARING은 기본값 EXACT이고, 별도로 명시하지 않는 한 세션 파라미터는 기본값이다.)

아래

— (구버전) 등록번호를 리터럴로 이어 붙여 발행 — 값마다 SQL 텍스트가 달라진다
 SELECT 축종, 품종, 등록일자 FROM 동물등록원부 WHERE 등록번호 = '410000112345678';
 SELECT 축종, 품종, 등록일자 FROM 동물등록원부 WHERE 등록번호 = '410000198765432';
 — ... 값만 바뀐 SELECT가 초당 수백 개 발행됨

— (개선) 등록번호를 바인드 변수로 발행
 SELECT 축종, 품종, 등록일자 FROM 동물등록원부 WHERE 등록번호 = :b1;

- ① 바인드 방식은 :b1 값이 달라도 SQL 텍스트가 동일하므로, 라이브러리 캐시에서 먼저 만들어 둔 커서(파싱된 실행계획)를 찾아 재사용하는 소프트웨어로 처리된다.
- ② 리터럴 방식이라도 오라클은 WHERE 절의 상수를 자동으로 정규화해 하나의 커서로 묶으므로, CURSOR_SHARING이 기본값이어도 값만 다른 두 SELECT는 같은 커서를 공유해 하드파싱이 일어나지 않는다.
- ③ CURSOR_SHARING을 FORCE로 바꾸면 리터럴 SELECT의 상수를 시스템 생성 바인드로 치환한 뒤 라이브러리 캐시를 뒤지므로, 값만 다른 SELECT들이 하나의 공유 커서로 수렴해 하드파싱이 줄어든다.
- ④ 텍스트가 조금이라도 다르면 — 대소문자·공백·주석이 달라도 — 오라클은 다른 SQL로 보고 별도 커서를 만들므로, 등록번호가 리터럴로 박힌 두 SELECT는 라이브러리 캐시에서 서로를 찾지 못해 각각 하드파싱된다.

14

비용 기반 옵티마이저가 매기는 Cost(비용)를 어떤 범위에서 비교에 쓸 수 있는지, 그리고 통계 유무가 Cost 산정에 관여하는 방식에 대한 설명으로 가장 적절한 것은?

- ① Cost는 옵티마이저가 I/O·CPU 같은 자원 소모를 하나의 수치로 환산해 매긴 값이므로, 서로 다른 두 SQL의 실행계획에 찍힌 Cost를 그대로 견주어도 된다. 같은 척도로 매겨진 숫자인 만큼, Cost가 더 낮은 쪽 SQL이 더 적은 자원으로 더 빨리 끝난다고 판단할 수 있다.
- ② Cost는 테이블 건수·컬럼 분포 같은 옵티마이저 통계를 재료로 삼아 산출되는 값이다. 따라서 통계가 없거나 낡아 재료가 부실하면 옵티마이저는 후보 계획의 Cost 자체를 계산할 수 없고, 우열을 가릴 수 없으니 실행계획을 세우지 못한 채 파싱 단계에서 오류를 낸다.
- ③ 실행계획에 나열된 오퍼레이션(라인)은 각 단계가 한 번씩 수행된다는 뜻이므로, 그 라인 개수를 더한 것이 곧 그 계획의 Cost다. 그래서 같은 SQL의 후보 계획을 견줄 때도 플랜 트리의 단계 수가 적은 쪽이 곧 더 낮은 Cost의 계획이다.
- ④ Cost는 하나의 SQL 안에서 여러 후보 실행계획의 우열을 가리기 위한 상대적 순위 지표이므로, 같은 SQL의 계획 A·B를 견주는 데는 쓸 수 있어도 서로 다른 SQL의 Cost를 맞대어 어느 쪽이 더 빠른지를 판단하는 근거로 삼기는 어렵다.

지적측량 성과 관리의 야간 배치가 당일 접수·처리된 측량성과(약 9,000만 행)를 스테이징에서 측량성과이력(약 15억 행)으로 대량 적재한다. 적재 시간을 줄이려고 아래처럼 APPEND 힌트와 병렬 DML을 적용했다. 측량성과이력 세그먼트는 NOLOGGING이고 DB는 force logging이 아니다. 이 대량 적재 튜닝에 대한 설명으로 가장 적절하지 않은 것은?

아 래

-- 당일 측량성과를 측량성과이력에 대량 적재하는 야간 배치
ALTER SESSION ENABLE PARALLEL DML;

INSERT /*+ APPEND PARALLEL(t, 4) */ INTO 측량성과이력 t
SELECT * FROM 성과_스테이징;

-- COMMIT 전, 같은 세션에서 적재 결과를 곧바로 SELECT 하려 시도한다

- ① INSERT /*+ APPEND */는 Direct Path Insert로 동작해, HWM 위쪽에 새 블록을 할당하며 데이터를 기록하고 버퍼 캐시와 freelist 탐색을 우회하므로 대량 적재가 빨라진다.
- ② ALTER SESSION ENABLE PARALLEL DML 후 병렬 INSERT를 수행하면 여러 PX 서버가 각자 자기 익스텐트에 direct path로 적재하며, 이때 대상 테이블은 배타 모드로 잠겨 커밋 전까지 같은 테이블에 대한 다른 세션의 DML은 대기한다.
- ③ 세그먼트가 NOLOGGING이면 direct path뿐 아니라 일반(conventional) 인서트에도 redo 절감이 적용되므로, 병렬 없이 일반 INSERT만 발행해도 데이터에 대한 redo가 사라져 대량 적재 부하가 크게 준다.
- ④ direct path로 적재한 세그먼트는 커밋 전에는 같은 트랜잭션이라도 다시 읽거나 수정할 수 없어, 위 배치가 커밋 전에 측량성과이력을 SELECT하면 ORA-12838이 발생한다 — 조회하려면 먼저 커밋해야 한다.

수령면허 발급 관리의 수령면허발급(약 4,800만 건)은 발급일자(DATE)를 기준으로 연 단위 RANGE 파티셔닝한 테이블이다(아래 DDL). 마지막 파티션 p_max는 VALUES LESS THAN (MAXVALUE)로 열려 있다. 각 선택지는 SELECT SUM(발급수수료) FROM 수령면허발급 WHERE ... 형태의 조회가 어떤 파티션에 접근하는지, 또는 새 연도 데이터가 어디에 저장되는지를 설명한다. Partition Pruning이 설명대로 작동한다고 볼 때 가장 적절한 것은?

아래

```
CREATE TABLE 수령면허발급 (
  발급번호      NUMBER,
  발급일자      DATE,
  면허종별      VARCHAR2(10),
  발급수수료    NUMBER,
  관할청코드    NUMBER
)
PARTITION BY RANGE (발급일자) (
  PARTITION p_2022 VALUES LESS THAN (DATE '2023-01-01'),
  PARTITION p_2023 VALUES LESS THAN (DATE '2024-01-01'),
  PARTITION p_2024 VALUES LESS THAN (DATE '2025-01-01'),
  PARTITION p_2025 VALUES LESS THAN (DATE '2026-01-01'),
  PARTITION p_max  VALUES LESS THAN (MAXVALUE)
);
```

- ① WHERE TO_CHAR(발급일자, 'YYYY') = '2024'는 파티션 키인 발급일자로 2024년을 겨냥한 조건이고, 그 연도 구간은 DDL의 p_2024 VALUES LESS THAN (DATE '2025-01-01') 경계와 그대로 맞아떨어진다. 따라서 p_2024 하나만 프루닝되어 그 파티션만 읽는다.
- ② WHERE 면허종별 = '제1종'은 VARCHAR2(10) 컬럼을 등치로 걸어 값이 하나로 확정된 조건이다. 등치로 범위가 확정되면 RANGE 프루닝이 걸리므로, 4,800만 건 전체를 훑지 않고 제1종이 몰려 있는 최근 연도 파티션 p_2024·p_2025만 골라 읽는다.
- ③ DDL에 나열된 마지막 연도 경계가 p_2025 VALUES LESS THAN (DATE '2026-01-01')이므로 2026년 발급분은 경계 밖의 새 연도다. 이때는 연 단위라는 간격 규칙에 따라 p_2026 파티션이 자동으로 새로 만들어지고, p_max는 비어 있는 채 2026년 데이터는 새 파티션에 저장된다.
- ④ WHERE 발급일자 >= DATE '2024-01-01' AND 발급일자 < DATE '2025-01-01'은 파티션 키에 가공이 없는 범위 조건이라 경계값과 대조되어 p_2024 파티션만 접근하고, 조회 대상이 아닌 p_2022·p_2023·p_2025·p_max는 스캔에서 제외된다.

17

열수송 누수 분석 야간 배치가 누수감지이벤트(약 3억 5천만 건)와 관로구간(약 4만 8천 건)을 관로구간ID로 조인해 구간별 누수 건수를 집계한다. 한쪽은 대형 팩트, 다른 쪽은 소형 차원 테이블이며, 어느 쪽도 관로구간ID 기준 파티션이 아니다. 아래 병렬 실행계획에서 두 테이블이 병렬 서버(Slave) 사이에 어떤 분배 방식으로 조인되는지 직접 지목한 설명으로 가장 적절한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10001			Q1,01	P->S
3	HASH GROUP BY				Q1,01	PCWP
* 4	HASH JOIN				Q1,01	PCWP
5	PX RECEIVE				Q1,01	PCWP
6	PX SEND BROADCAST	:TQ10000			Q1,00	P->P
7	PX BLOCK ITERATOR				Q1,00	PCWC
* 8	TABLE ACCESS FULL	관로구간			Q1,00	PCWP
9	PX BLOCK ITERATOR				Q1,01	PCWC
* 10	TABLE ACCESS FULL	누수감지이벤트			Q1,01	PCWP

- ① 조인 키 관로구간ID의 해시로 관로구간(Id 8)과 누수감지이벤트(Id 10)를 양쪽 모두 재분배(hash-hash)한 뒤 각 서버가 자기 버킷 범위의 행만 조인하며, 그 재분배가 Id 6에서 관로구간, Id 9에서 누수감지이벤트에 대해 각각 일어난다.
- ② 소형 관로구간(Id 8)을 PX SEND BROADCAST(Id 6)로 모든 병렬 서버에 복제하고, 대형 누수감지이벤트(Id 10)는 재분배 없이 각 서버가 자기 블록 범위만 읽어(Id 9 PX BLOCK ITERATOR) 로컬로 조인하는 broadcast 분배다.
- ③ 두 테이블이 관로구간ID로 동일하게 파티셔닝돼 있어 재분배 없이 대응 파티션 집합만 로컬로 조인하는 (full) 파티션 와이즈 조인이며, Id 8·Id 10이 같은 PX PARTITION 아래에서 읽힌다.
- ④ 대형 누수감지이벤트를 PX SEND BROADCAST로 전 서버에 복제하고, 소형 관로구간은 각 서버가 자기 블록 범위만 읽어 조인하는 broadcast 분배다.

18

소트 계열 오퍼레이션(SORT ORDER BY·SORT GROUP BY·SORT UNIQUE·SORT JOIN)이 도는 이유와, 인덱스의 정렬된 순서로 그 소트가 생략되는 범위에 대한 설명으로 가장 적절하지 않은 것은?

- ① SORT ORDER BY는 결과 정렬을, SORT UNIQUE는 DISTINCT·집합 연산의 중복 제거를, SORT JOIN은 소트 머지 조인에서 양쪽 입력을 조인 키로 정렬하려고 도는 등, 소트 계열 오퍼레이션은 저마다 도는 이유가 다르다.
- ② 소트 머지 조인의 SORT JOIN은 양쪽 입력을 각자 조인 키 순서로 정렬해야 병합할 수 있으므로, 한쪽 입력이 조인 키로 정렬된 인덱스를 통해 이미 정렬된 순서로 공급되면 그쪽의 SORT JOIN 단계는 생략될 수 있다.
- ③ SORT GROUP BY는 그룹 키로 정렬하며 집계하므로, 그룹 키가 인덱스 선두 컬럼의 정렬 순서와 일치하면 정렬 없이 인덱스를 훑는 것으로 대체돼 그 소트가 생략될 수 있다.
- ④ 소트 머지 조인에서 조인 키에 인덱스가 하나라도 있으면 그 정렬 순서를 양쪽 입력이 함께 이용하므로, 인덱스가 걸린 조인 키로 조인하면 양쪽 SORT JOIN이 둘 다 사라진다.

19

자원봉사 시간은행의 봉사활동실적(약 2,600만 건)에 봉사단체 가·나·다의 누적봉사시간이 각각 100·200·300(합 600)으로 들어 있다. 갱신 세션 TX1이 두 건의 UPDATE를 발행하되 하나만 커밋하고, 그 뒤 조회 세션 TX2가 SELECT SUM(누적봉사시간)을 한 번 발행한다(아래 타임라인). TX2의 SELECT(t3)가 반환하는 SUM(누적봉사시간)으로 적절한 것은? (단, 오라클이며 격리수준은 기본값 READ COMMITTED이고, 관련 Undo는 정상적으로 남아 있다.)

아래		
시각	TX1 (갱신 세션)	TX2 (조회 세션)
t1	UPDATE 봉사활동실적 SET 누적봉사시간=+60 WHERE 단체='나'; (나 200→260) COMMIT;	— 커밋 완료
t2	UPDATE 봉사활동실적 SET 누적봉사시간=+50 WHERE 단체='다'; (다 300→350)	— 미커밋 (커밋 안 함)
t3		SELECT SUM(누적봉사시간) FROM 봉사활동실적; (이 값을 구함)
초기: 가=100, 나=200, 다=300 (합 600)		

- ① 660 — 커밋된 '나'의 변경(260)만 반영하고, 미커밋인 '다'는 커밋 전 값(300)으로 읽는다.
- ② 710 — t1에 커밋된 '나'(260)와 t2에서 갱신한 '다'(350)를 함께 반영해 100+260+350으로 읽는다.
- ③ 600 — t3 이전에 t1에서 커밋된 '나'의 +60까지 무시하고 초기 스냅샷 100+200+300을 그대로 읽는다.
- ④ 650 — '나'는 커밋 전 값 200으로, '다'는 t2의 변경값 350으로 읽어 100+200+350이 된다.

20

평생학습관 강의실배정 시스템(SQL Server)의 강의실배정현황(약 6,200만 건)은 기본 READ COMMITTED(잠금 기반, RCSI OFF)로 운영한다. 세션 53과 세션 57이 각각 서로 다른 강의실 행을 먼저 갱신해 X 잠금을 뒀 뒤, 이어서 상대가 뒀 강의실 행을 갱신하려 한다. 이때 sys.dm_tran_locks 등을 조회한 결과가 아래와 같다. 이 상황에 대한 해석으로 가장 적절한 것은?

아래				
[sys.dm_tran_locks / 요청 상태 - 강의실배정현황 (RCSI = OFF)]				
session_id	request_mode	resource(KEY)	request_status	대기 대상
53	X	강의실:R100	GRANT	-
53	X	강의실:R200	WAIT	→ 57 보유분 대기
57	X	강의실:R200	GRANT	-
57	X	강의실:R100	WAIT	→ 53 보유분 대기
* KEY = 행(키) 잠금, X = 배타 잠금				
* 53은 R100 보유 · R200 대기, 57은 R200 보유 · R100 대기				

- ① 53이 57을 기다리고 있으므로 단순 블로킹이다. 57이 트랜잭션을 커밋하면 53의 R200 대기가 자동으로 풀려 두 세션 모두 정상 종료된다.
- ② 두 세션이 서로를 기다리는 교착이지만, SQL Server는 교착을 스스로 해소하지 못하므로 두 세션은 무한 대기 상태로 남고 관리자가 한쪽을 KILL해야 한다.
- ③ 53은 R200을, 57은 R100을 서로 기다리는 순환 대기라 교착 상태이며, SQL Server의 교착 감시가 이를 감지해 롤백 비용이 작은 쪽을 victim(오류 1205)으로 선정·롤백하고 상대 세션은 진행한다.
- ④ 두 세션이 함께 강의실 KEY에 X 잠금을 보유하고 있다는 사실은 잠금 경합일 뿐 교착의 근거가 아니므로, 먼저 잠금을 획득한 53이 우선권을 얻어 R200까지 확보하며 진행한다.

정답 및 해설

■ 정답 모아보기

1. ②	2. ①	3. ③	4. ④	5. ③
6. ①	7. ③	8. ①	9. ①	10. ②
11. ②	12. ④	13. ②	14. ④	15. ③
16. ④	17. ②	18. ④	19. ①	20. ③

문제 1. 정답 ②

클러스터링 팩터가 인덱스 쪽(리프 정렬)에 작용하는 값인지, 테이블 쪽(블록 방문)에 작용하는 값인지를 갈라 봐야 합니다.

인덱스 경우 테이블 액세스 = 두 단계의 비용

(1) 인덱스 수직 탐색 + 리프 스캔 ← 인덱스 높이 · 리프 블록 수가 좌우

(2) ROWID로 테이블 블록 랜덤 액세스(Single Block I/O) ← 클러스터링 팩터가 좌우

클러스터링 팩터(CF)가 재는 것

= 인덱스 순서대로 테이블을 읽을 때 방문하게 되는 '테이블 블록' 수

→ CF는 (2) 테이블 랜덤 액세스의 블록 방문 횟수를 좌우하는 값이다

클러스터링 팩터는 인덱스 정렬 순서를 따라 테이블을 읽을 때 방문하게 되는 테이블 블록 수의 척도입니다. 즉 리프의 ROWID로 테이블 블록을 찾아가는 (2) 랜덤 액세스 단계의 방문 횟수를 좌우하는 값이며, 리프 엔트리가 리프 블록 안에 얼마나 정렬돼 담겼는지(인덱스 내부의 정렬 밀도)를 재는 값이 아닙니다.

②는 이 작용점을 정반대로 뒤집었습니다. 클러스터링 팩터를 "인덱스 리프의 정렬 밀도를 나타내는 값이라 인덱스 스캔 비용만 좌우하고, 테이블 랜덤 액세스의 블록 방문과는 무관하다"고 했지만, 클러스터링 팩터가 정작 좌우하는 것은 바로 그 테이블 블록 랜덤 액세스 방문 횟수입니다. CF가 작용하는 단계를 (2)에서 (1)로 옮겨 서술한 것입니다.

①·③·④는 옳습니다. 비용의 상당 부분이 행별 테이블 랜덤 액세스에서 나온다는 점(①), CF가 좋으면 인접 ROWID가 같은 블록에 몰려 방문 블록 수가 준다는 점(③), CF가 나쁘면 넓은 범위 조회에서 Full Table Scan보다 불리해질 수 있다는 점(④)이 모두 랜덤 액세스와 CF의 실제 관계에 부합합니다.

선택지	판정	이유
①	○	인덱스 경우 액세스의 큰 비용은 ROWID로 테이블 블록을 건별로 찾아가는 Single Block I/O이며, 결과 행이 늘수록 이 테이블 랜덤 액세스 횟수가 비례해 증가합니다
②	×	클러스터링 팩터는 인덱스 순서로 테이블을 읽을 때의 테이블 블록 방문 횟수, 곧 랜덤 액세스 방문 수를 좌우하는 값입니다. 이를 "인덱스 리프 정렬 밀도라 테이블 방문과 무관하다"고 한 것은 CF의 작용점을 정반대로 옮긴 서술입니다
③	○	CF가 좋아 인접 ROWID가 같은 블록에 모이면 한 번 읽은 블록에서 여러 행을 처리해, Single Block I/O로 방문하는 테이블 블록 수가 줄어듭니다
④	○	인덱스가 잘 정렬돼 있어도 테이블 저장 순서가 어긋나 CF가 나쁘면 행마다 다른 블록을 방문해, 넓은 범위 조회에서는 Full Table Scan보다 불리해질 수 있습니다

■ 이 문제의 핵심

1. 인덱스 경우 테이블 액세스 비용은 인덱스 탐색 · 리프 스캔과 ROWID 테이블 랜덤 액세스(Single Block I/O)의 두 단계로 나뉩니다.
2. 클러스터링 팩터는 인덱스 순서로 테이블을 읽을 때 방문하는 테이블 블록 수의 척도로, 랜덤 액세스 방문 횟수를 좌우합니다.
3. CF가 좋으면 인접 ROWID가 같은 블록에 몰려 Single Block I/O 방문 블록 수가 줄어듭니다.
4. 인덱스가 아무리 잘 정렬돼 있어도 CF가 나쁘면 넓은 범위 조회에서 Full Table Scan보다 불리할 수 있습니다.

■ 한 줄 정리 클러스터링 팩터는 리프의 ROWID로 테이블 블록을 찾아가는 랜덤 액세스의 방문 횟수를 좌우하는데, "인덱스 리프 정렬 밀도라 테이블 방문과 무관하다"는 ②는 CF의 작용점을 인덱스 쪽으로 뒤집은 서술입니다.

문제 2. 정답 ①

인덱스가 어떤 블록들로 이뤄진 세그먼트인지, 'B-Tree'의 B가 무엇인지를 정확히 짚어야 합니다.

B-Tree 인덱스 = 하나의 세그먼트

루트 블록 : 1개 (최상단)

브랜치 블록 : 중간 층 (하위 블록의 경계값 + 포인터)

리프 블록 : 최하단 (인덱스 키 + ROWID, 좌우 연결)

탐색 : 루트 → 브랜치 → 리프 (수직 탐색)

높이 : 루트~리프 레벨 수 (보통 2~4로 낮게 유지)

B : Balanced(균형) - 어느 리프든 같은 깊이. Binary(이진)가 아님

한 블록에 엔트리가 가득 차면 split으로 공간을 나눠 균형 유지

B-Tree 인덱스는 그 자체가 하나의 세그먼트로, 루트·브랜치·리프 블록이 모두 그 세그먼트 안에 함께 저장됩니다. 조회는 루트에서 브랜치를 거쳐 리프로 내려가는 수직 탐색으로 시작 지점을 찾고, 어느 리프에 도달하든 거치는 레벨 수(인덱스 높이)가 같도록 균형이 유지됩니다. 한 블록이 가득 차면 블록 분할로 공간을 나눠 이 균형을 지킵니다. ①은 이 구조를 그대로 서술해 옳습니다.

②는 'B=Binary'라는 반사적 연상에 걸린 서술입니다. B-Tree의 B는 균형(Balanced)을 뜻하며, 루트든 브랜치든 한 블록은 자식을 둘이 아니라 수십~수백 개까지 가리킬 수 있습니다(높은 팬아웃). 그래서 인덱스 높이는 리프 수의 밑 2 로그를 따라 깊어지는 것이 아니라 훨씬 완만하게 늘어, 대량 데이터에서도 2~4 수준으로 낮게 유지됩니다.

③은 루트·브랜치가 세그먼트 밖에 따로 있다는 잘못된 서술입니다. 루트·브랜치·리프는 모두 같은 인덱스 세그먼트에 디스크로 저장되므로 수직 탐색 구간이 메모리 안에서만 끝나지 않고, 블록 분할도 리프에서만 일어나는 것이 아니라 상위 블록이 가득 차면 브랜치·루트로 전파돼 높이가 한 단계 올라가기도 합니다. ④는 일반 B-Tree 인덱스를 IOT·클러스터형 인덱스와 혼동한 서술로, 일반 인덱스의 리프에는 행 전체가 아니라 키와 ROWID만 담기므로 인덱스에 없는 컬럼을 읽으려면 테이블을 되짚어야 하고, 인덱스 세그먼트도 테이블과 별개입니다.

선택지	판정	이유
①	○	B-Tree 인덱스는 루트·브랜치·리프로 이뤄진 하나의 세그먼트이며, 수직 탐색으로 시작 리프를 찾고 블록 분할로 균형(낮은 높이)을 유지합니다
②	×	B-Tree의 B는 Balanced(균형)이며 루트·브랜치는 자식을 둘씩만이 아니라 수십~수백 개까지 가리킵니다(높은 팬아웃). 높이가 리프 수의 밑 2 로그에 비해 가파르게 깊어진다는 것은 'B=Binary' 반사에 따른 오해로, 실제 높이는 2~4 수준에 머물러 있습니다
③	×	루트·브랜치·리프는 모두 같은 인덱스 세그먼트에 디스크로 저장되므로 수직 탐색 구간이 메모리 안에서만 끝나지 않습니다. 블록 분할도 리프 전용이 아니라 상위 블록이 가득 차면 브랜치·루트로 전파돼 높이가 한 단계 올라가기도 합니다
④	×	일반 B-Tree 인덱스 리프에는 키와 ROWID만 담기고 인덱스는 테이블과 별개 세그먼트입니다. 리프에 행 전체가 담겨 테이블을 되짚지 않아도 된다는 것은 IOT·클러스터형 인덱스와 혼동한 서술입니다

▪ 이 문제의 핵심

1. B-Tree 인덱스는 루트·브랜치·리프 블록이 모두 담긴 하나의 세그먼트입니다.
2. 조회는 루트→브랜치→리프의 수직 탐색으로 시작 리프를 찾고, 인덱스 높이는 낮게(보통 2~4) 균형 유지됩니다.
3. B-Tree의 B는 Balanced(균형)이며 Binary가 아닙니다 — 브랜치는 자식을 여럿 가리키는 높은 팬아웃 구조입니다.
4. 일반 인덱스 리프에는 키+ROWID만 담겨, 인덱스에 없는 컬럼은 테이블을 되짚어야 연됩니다(IOT·클러스터형과 다름).

▪ 한 줄 정리 B-Tree 인덱스는 루트·브랜치·리프가 담긴 균형 세그먼트라는 ①이 옳고, 'B=이진'이라 자식이 둘씩·높이가 밑 2 로그라는 ②는 키워드 반사에 걸린 오해입니다.

문제 3. 정답 ③

동적 SQL이 실행마다 리터럴 값을 이어 붙이므로 SQL 텍스트가 매번 달라지고, 커서는 라이브러리 캐시에서 공유되지 못합니다. 그러면 파싱은 대부분 하드파싱이 됩니다. 먼저 하드파싱율을 쟁니다.

하드파싱율 = 라이브러리 캐시 미스(비재귀) ÷ 비재귀 Parse 콜
= 485,000 ÷ 500,000 = 97.0%

→ 거의 매 실행이 계획을 새로 생성하는 하드파싱

하드파싱은 계획을 새로 만들 때마다 데이터 덱서리(권한·통계·객체 정보)를 조회하는 재귀 statement를 대량으로 유발합니다. 재귀 축을 봅시다.

재귀 Call 비율 = 재귀 Total 콜 ÷ 비재귀 Total 콜 = 5,000,000 ÷ 1,000,000 = 5배

재귀 비중 = 5,000,000 ÷ (5,000,000 + 1,000,000) = 83.3%

→ 하드파싱이 덱서리 재귀 SQL 500만 콜을 낳았다(하드파싱의 부산물)

시간의 정체도 파싱 속입니다. 비재귀 Parse에 CPU 40.0초가 잡혀 있습니다.

Parse CPU 비중(비재귀) = 40.0 ÷ 50.0 × 100 = 80.0% ← 계획 생성(하드파싱) CPU

Execute 비중 = $8.0 \div 50.0 \times 100 = 16.0\%$ ← 실제 갱신은 소량

여기서 두 축을 분리해야 합니다.

축 A: 하드파싱(계획 생성 · 라이브러리 캐시 MISS · 재귀 디셔너리 콜) — 미스 97.0%, 재귀 5배 → 병목
 축 B: 파싱 콜 횟수(소프트파싱 · 커서 미보유) — Parse:Execute=1:1이지만 여기서 소프트웨어가 아닌

동적 SQL이 리터럴로 텍스트를 매번 바꾸니 커서가 공유되지 않는다. Parse 콜을 캐싱으로 붙잡아도(축 B 처방)
 텍스트가 다른 커서는 재사용되지 않는다. 원인은 축 A(하드파싱)이므로 처방도 축 A라야 한다.

처방은 커서 홀드가 아니라 동적 SQL의 리터럴을 바인드 변수로 바꿔 SQL 텍스트를 하나로 고정하는 것입니다. 텍스트가 같아지면 커서가 공유돼 하드파싱이 소프트웨어로 바뀌고, 디셔너리 재귀 콜(500만)과 Parse CPU(40초)가 함께 사라집니다. ③이 하드파싱 축(계획 생성·재귀 콜)을 정확히 지목했습니다.

선택지	판정	이유
①	×	재귀 Query 800만+1,200만 = 2,000만은 Disk 0의 논리 읽기라 이미 캐시에서 나오고 있어 버퍼 캐시를 키워도 줄지 않습니다. 재귀 statement와 그 CPU 21.0초는 하드파싱이 부른 디셔너리 조회, 곧 원인이 아니라 결과이므로 하드파싱이 계속되는 한 재귀 콜은 다시 발생합니다. 뿌리(하드파싱)가 아니라 부산물을 겨냥한 헛짚음입니다
②	×	Parse 500,000콜과 Parse CPU 40.0초를 소프트웨어로 본 것이 오류입니다. 라이브러리 캐시 미스가 485,000건으로 미스율 97.0%라 이 파싱들은 소프트웨어가 아니라 하드파싱이고, 리터럴로 텍스트가 매번 달라 커서 홀드· <code>session_cached_cursors</code> 로 붙잡을 공유 커서 자체가 없습니다. 소프트웨어 빈도 축을 하드파싱 축으로 오인한 별개 축 혼동입니다
③	○	하드파싱율 $485,000/500,000 = 97.0\%$ 로 계획 생성 축이 병목임을 짚고, 그 하드파싱이 재귀 디셔너리 콜 500만(비재귀의 5배)을 유발했음을 연결한 뒤, 동적 SQL의 리터럴을 바인드 변수로 바꿔 텍스트를 고정하는 처방이 정확합니다
④	×	Execute CPU 6.0초·Elapsed 8.0초는 전체 50초의 16.0%뿐이고, 갱신 1건당 Query 2블록·Current 1블록은 이미 최소 수준의 논리 읽기입니다. 병목은 실행 축이 아니라 Parse CPU 40초(80.0%)의 하드파싱 축이므로, UPDATE 실행계획을 다시 짜도 파싱 지연은 그대로 남습니다

■ 이 문제의 핵심

- 하드파싱율 = 라이브러리 캐시 미스 ÷ Parse 콜. 여기서는 $485,000/500,000 = 97.0\%$ 로, 동적 SQL이 리터럴을 붙여 텍스트가 매번 달라 커서가 공유되지 못해 사실상 매 실행이 하드파싱입니다.
- 하드파싱은 재귀 Call을 낳습니다. 계획을 새로 만들 때마다 데이터 디셔너리를 조회하므로 재귀 statement가 500만 콜(비재귀의 5배)로 폭증했습니다 — 재귀 축은 하드파싱의 부산물입니다.
- 하드파싱 축(계획 생성)과 소프트웨어 빈도 축(커서 미보유)은 별개입니다. Parse:Execute = 1:1이어도 미스율이 97.0%면 커서 캐싱은 붙잡을 공유 커서가 없어 무력합니다.
- 처방은 커서 홀드·버퍼 캐시·실행계획 튜닝이 아니라 동적 SQL의 리터럴 → 바인드 변수입니다. 텍스트가 하나로 고정돼야 커서가 공유됩니다.
- Parse CPU 40초(80.0%)가 시간을 지배하고 Execute는 16.0%뿐이라, 병목은 실행이 아니라 파싱(계획 생성)에 있습니다.

■ 한 줄 정리 하드파싱율이 97.0%(485,000/500,000)이고 재귀 디셔너리 콜이 500만(비재귀의 5배)으로 폭증한 것은 동적 SQL의 리터럴 때문에 커서가 공유되지 못해 매 실행이 하드파싱이기 때문이며, 처방은 커서 홀드·버퍼 캐시가 아니라 리터럴을 바인드 변수로 바꿔 하드파싱을 소프트웨어로 돌리는 것이다.

문제 4. 정답 ④

계획을 '읽는 규칙' 세 가지 — 실행 순서, Predicate 두 조건의 역할, format 수준 — 를 각각 정확히 짚어야 합니다.

실행 순서 : 위→아래(X). 들여쓰기 부모-자식(0)
 가장 깊은 자식부터 수행 → 결과를 부모로 올림
 Predicate : access 조건 = 진입 탐색 범위를 좁힘(시작·종료점)
 filter 조건 = 얻은 행을 추가로 걸러 냄(범위는 그대로)
 format 수준 : BASIC = Operation·Name만 (Predicate·Cost 생략)
 TYPICAL = 기본, Cost·카디널리티 포함
 ALL = Predicate 섹션까지 상세

실행계획 트리에서 단계의 수행 순서는 적힌 차례가 아니라 들여쓰기가 드러내는 부모-자식 관계로 읽습니다. 가장 안쪽 자식이 먼저 수행되고 그 결과가 부모로 올라가는 식입니다. 또 Predicate 섹션의 access 조건은 탐색 범위를 좁히는 조건이고 filter 조건은 얻은 행을 추가로 걸러 내는 조건이라 역할이 다릅니다. 이 둘을 구분해 읽어와 "인덱스로 넓게 훑어 놓고 filter로 대량을 버리는" 스캔 범위 문제를 짚어낼 수 있습니다. ④는 이 두 규칙을 정확히 서술해 옳습니다.

나머지는 모두 계획을 읽는 규칙의 경계를 뚫는 서술입니다. ①은 Id 0부터 차례로 매겨진다는 사실을 실행 순서로 옮겨 붙여 "위에서 아래 그대로"라 했지만, Id는 계획 라인의 식별 번호일 뿐 수행 차례가 아니며 순서는 들어쓰기의 부모-자식 관계로 정해집니다 — 들어쓰기를 장식이라 한 것은 그 경계를 지운 서술입니다. ②는 Predicate 섹션이 WHERE 조건을 단계별로 나열한다는 데까지는 맞지만 access와 filter를 같은 역할로 뚫었는데, filter는 탐색 범위를 좁히지 못하고 이미 얻은 행을 걸러 낼 뿐입니다. ③은 'BASIC'이 Cost·카디널리티·Predicate까지 가장 상세히 보여 준다고 했지만, 'BASIC'은 오히려 그것들을 모두 생략하고 Operation·Name만 남기는 최소 수준이라 조건을 보려면 'ALL' 등으로 올려야 합니다.

선택지	판정	이유
①	×	Id가 0부터 차례로 매겨지는 것은 맞지만 그것은 계획 라인의 식별 번호일 뿐 수행 차례가 아닙니다. 실행 순서는 들어쓰기 부모-자식 관계로 읽어 가장 깊은 자식부터 수행되므로, 들어쓰기를 장식으로 본 것은 읽기 규칙의 경계를 뚫는 서술입니다
②	×	Predicate 섹션이 WHERE 조건을 단계 번호와 함께 나열하는 것은 맞지만, access는 탐색 범위를 좁히는 조건이고 filter는 그렇게 얻은 행을 걸러 내는 조건이라 역할이 다릅니다. 둘을 같은 일로 보아 갈라 볼 필요가 없다고 한 것은 경계 오해입니다
③	×	기본값이 'TYPICAL'인 것은 맞으나 'BASIC'은 거기서 Cost·카디널리티·Predicate 섹션을 걷어 내고 Operation·Name만 남기는 최소 수준입니다. access·filter를 보려면 'BASIC'이 아니라 'ALL' 등으로 올려야 합니다
④	○	실행 순서는 들어쓰기 부모-자식으로 읽어 깊은 자식부터 수행하며, access(범위 축소)와 filter(행 제거)를 구분해 읽어야 스캔 범위 문제를 짚을 수 있습니다

- 이 문제의 핵심
- 1. 계획 트리의 실행 순서는 적힌 차례가 아니라 들어쓰기 부모-자식 관계로, 가장 깊은 자식부터 수행됩니다.
- 2. Predicate 섹션의 access 조건은 탐색 범위를 좁히고, filter 조건은 얻은 행을 추가로 걸러 냅니다(범위는 그대로).
- 3. format을 'BASIC'으로 낮추면 Operation·Name만 남고 Cost·카디널리티·Predicate가 생략됩니다.
- 4. EXPLAIN PLAN+DBMS_XPLAN.DISPLAY로 보는 것은 실행 전 예상(추정) 실행계획입니다.
- 한 줄 정리 실행 순서는 들어쓰기 부모-자식으로 읽고 access·filter를 구분해 본다는 ④가 옳고, 위에서 아래 순서(①)·두 조건 동일시(②)·BASIC이 가장 상세(③)는 계획 읽기 규칙의 경계를 뚫는 서술입니다.

문제 5. 정답 ③

먼저 BCHR로 논리 대비 물리 읽기를 봅시다.

BCHR = ((Query + Current) - Disk) / (Query + Current) × 100
= (3,000,000 + 0 - 180,000) / 3,000,000 × 100
= 2,820,000 / 3,000,000 × 100 = 94.0%
물리 읽기 비중 = 180,000 / 3,000,000 × 100 = 6.0%

캐시 적중이 94%로 높습니다. 논리 읽기 300만 중 실제 디스크에서 올라온 것은 18만(6%)뿐입니다. 그런데도 느립니다 — 이때는 물리 I/O가 아니라 논리 읽기 자체의 양을 의심해야 합니다. 시간의 정체를 봅시다.

CPU 비중 = 36.000 / 45.000 × 100 = 80.0% → CPU가 시간을 지배
대기 시간 = 45.000 - 36.000 = 9.0초 (20%) ← 그중 물리 읽기 대기

응답의 80%가 CPU입니다. 물리 읽기가 작는데 CPU가 큰 것은, 캐시에 있는 블록을 반복해서 논리적으로 읽느라(buffer get) CPU를 태운다는 신호입니다. 어디서 났는지 Row Source로 찾되, cr(논리)과 pr(물리)을 분리합니다.

pw(temp spill) = 모든 노드 0 → spill 없음
논리 읽기(cr) 정체 = NL 조인의 영사업무접수 반복 방문 = 인덱스 재탐색 1,900,000 + 테이블 랜덤 self 1,099,970
(테이블 self = TABLE ACCESS BY INDEX ROWID 2,999,970 - 자식 INDEX 1,900,000 = 1,099,970)
→ NL 조인이 드라이빙 건마다 (인덱스+테이블) 방문을 반복해 150만 행마다 블록을 논리적으로 읽음 → cr 300만
물리 읽기(pr) = 180,000 (영사업무접수), 그 노드 time의 대부분은 물리 대기가 아니라 논리 읽기 CPU

pw=0이라 spill은 없고, pr은 18만으로 작습니다. 정체는 NL 조인이 영사업무접수를 150만 번 방문하며 인덱스 재탐색(cr 1,900,000)과 테이블 랜덤 액세스(self 1,099,970)로 쌓은 논리 읽기 300만이고, 그 대부분이 캐시 적중이라 디스크가 아닌 CPU를 태웁니다.

BCHR 94% 높음 → 물리 디스크는 병목이 아님(Disk 6%)
CPU 80% 지배 → 논리 읽기(buffer get) 300만이 CPU를 소진 → 논리 I/O 바운드

따라서 처방은 물리 읽기가 아니라 논리 읽기 횟수 자체를 줄이는 것 — NL 조인을 해시 조인으로 바꾸거나 액세스 경로를 개선해 영사업무접수 방문 횟수를 낮추는 것입니다. ③이 BCHR과 병목 지목을 함께 옳게 했습니다.

선택지	판정	이유
①	×	pr 180,000이 영사업무접수 노드에 몰린 것은 맞지만, 그 노드 time 36초의 대부분은 물리 대기가 아니라 cr 2,999,970을 훑는 논리 읽기 CPU입니다. 물리는 논리 읽기의 6%뿐이고 BCHR도 94%라, 버퍼 캐시를 더 키워 이 6%를 없애도 CPU 80%를 만드는 논리 읽기 300만은 그대로 남습니다
②	×	Elapsed-CPU = 45.000-36.000 = 9.0초는 전체의 20%뿐이고 시간의 80%는 CPU입니다. 9초를 I/O 바운드 근거로 삼아 DOP를 올리면 물리 읽기 18만만 나뉘 질 뿐, CPU를 태우는 논리 읽기 300만은 그대로여서 병목을 물리 대기로 오귀속한 진단입니다
③	○	BCHR (3,000,000-180,000)/3,000,000 = 94.0%로 캐시 적중이 높아 물리 읽기가 6%뿐임을 짚고, CPU 80%·pw=0·cr 300만이 영사업무접수 반복 방문에 쌓였음을 연결해 병목을 논리 읽기(buffer get)로 지목하고 논리 읽기 축소를 처방한 것이 정확합니다
④	×	NESTED LOOPS가 1,500,000행에 time 44초를 쓴 것은 맞지만 그 노드는 pw=0이라 temp로 흘린 spill이 없습니다. NL 조인은 결과를 모아 두지 않고 건건이 흘러보내므로 대량 작업 영역이 개입하지 않아, PGA를 키워도 45초는 그대로입니다 — 논리 읽기 CPU를 spill로 오귀속한 것입니다

▪ 이 문제의 핵심

1. BCHR = ((Query+Current)-Disk)/(Query+Current) × 100. 여기서는 (3,000,000-180,000)/3,000,000 = 94.0%로 캐시 적중이 높아, 논리 읽기의 6%만 디스크에서 올라왔습니다.
2. BCHR이 높은데도 느리면 논리 읽기 양을 의심합니다. 물리 I/O가 작는데 CPU가 80%면, 캐시에 있는 블록을 반복 논리 읽기(buffer get)하며 CPU를 태우는 논리 I/O 바운드입니다.
3. cr과 pr·pw를 분리해야 합니다. cr 300만은 NL 조인이 **영사업무접수**를 150만 번 방문해 쌓은 것이고, pr은 18만·pw는 0입니다.
4. 높은 BCHR이 좋은 신호만은 아닙니다. 불필요한 논리 읽기가 많아도 캐시에서 나오면 BCHR은 높게 뜨므로, 병목은 물리가 아니라 논리 읽기 횟수일 수 있습니다.
5. 처방은 논리 읽기 축소(NL→해시 조인 전환·액세스 경로 개선)입니다. 버퍼 캐시·DOP·PGA는 CPU를 태우는 논리 I/O를 근거부터 잘못 겨냥합니다.

▪ 한 줄 정리 BCHR 94%로 캐시 적중이 높아 물리 읽기가 6%뿐이고 pw=0이라 spill도 없는데 CPU가 80%인 것은 NL 조인이 **영사업무접수**를 150만 번 방문해 논리 읽기 300만을 쌓았기 때문이므로 병목은 논리 I/O(buffer get)이고, 이를 물리 읽기·DOP·집계 spill로 돌리면 원인을 오귀속하게 된다.

문제 6. 정답 ①

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 관찰_IX Card 6,300 ← 인덱스에서 훑고 남은 건수
TABLE ACCESS BY INDEX ROWID 표본관찰기록 Card 1,650 ← 테이블까지 거친 뒤 남은 건수

관찰_IX는 (수목원코드, 관찰일자) 순서입니다. WHERE에는 수목원코드 = 'K12'(등치)와 관찰일자 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 연속해서 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 6,300건입니다.

표본상태는 **관찰_IX**의 구성 칼럼이 아닙니다. 인덱스에 값이 없으니 인덱스 단계에서는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 6,300건이 이 단계를 거치며 1,650건으로 줄어듭니다.

6,300 (인덱스 액세스: 수목원코드=, 관찰일자 범위)
└─ 테이블 필터(표본상태='고사') 적용 → 1,650 (최종)

즉 ①이 이 구조를 정확히 서술합니다. 표본상태까지 인덱스 액세스 조건으로 끌어올린 ②는 그 조건이 범위를 좁혔다면 Id 3 Card가 이미 1,650이어야 한다는 점에서, 선두 칼럼 등치까지 스캔 후 필터로 밀어 넣은 ③은 그랬다면 Id 3이 RANGE SCAN으로 Cost 21에 끝날 수 없다는 점에서, 인덱스 반환 건수 6,300을 최종 건수로 본 ④는 Id 2 Card가 1,650이라는 점에서 각각 이 Card 흐름과 어긋납니다.

선택지	판정	이유
①	○	수목원코드(등치)·관찰일자(범위)는 인덱스 액세스 조건, 표본상태는 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 6,300→1,650으로 줄어드는 흐름과 일치합니다
②	×	표본상태는 관찰_IX 구성 칼럼이 아니므로 인덱스 액세스 조건이 될 수 없습니다. 테이블 필터인 표본상태를 인덱스 액세스 조건 집합에 끼워 넣은 진술로, 그것이 Id 3에서 스캔 범위를 좁혔다면 INDEX RANGE SCAN Card가 이미 1,650이었을 텐데 실제로는 6,300입니다
③	×	두 번째 칼럼이 범위라는 성질을 선두 칼럼에까지 확대한 진술입니다. 선두 칼럼 등치는 그 자체로 액세스 조건이고, 범위는 그 다음 칼럼부터 적용됩니다. 등치까지 필터라 리프 전 구간을 훑었다면 Id 3은 INDEX RANGE SCAN이 아니라 전체 스캔이 됐을 것이고 Cost도 21에 그치지 않습니다
④	×	6,300은 Id 3 인덱스 단계 Card일 뿐이고, 표본상태 필터를 거친 Id 2의 Card는 1,650입니다. 컬럼만 채워 넣는 것이 아니라 테이블 액세스 단계에서 건수 감소가 분명히 일어납니다

■ 이 문제의 핵심

- 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 수목원코드(=)·관찰일자(범위)가 INDEX RANGE SCAN 구간을 만듭니다(Card 6,300).
- 인덱스에 없는 칼럼은 테이블 필터. 표본상태는 ROWID로 테이블을 읽은 뒤 TABLE ACCESS BY INDEX ROWID 단계에서 걸러집니다.
- 두 노드의 Card 차이(6,300 vs 1,650)가 곧 테이블 필터의 효과입니다. 인덱스 반환 건수 ≠ 최종 건수.
- 테이블 필터 조건을 인덱스 액세스 조건 집합에 끼워 넣지 않습니다. 인덱스에 없는 표본상태는 스캔 범위를 정하는 데 관여하지 못합니다.

■ 한 줄 정리 선두 칼럼 등치(수목원코드)와 다음 칼럼 범위(관찰일자)는 인덱스 액세스 조건이 되고, 인덱스에 없는 표본상태는 테이블 액세스 단계 필터가 되어 Card가 6,300에서 1,650으로 줄어든다.

문제 7. 정답 ③

세 지표를 차례로 계산해 두 가설("왕복이 병목"·"물리 읽기가 병목")을 각각 검증한 뒤 병목을 종합합니다.

① 배열 크기(Array Processing). 마지막 한 번은 "더 없음(no-more-data)" 인출이라 -1 합니다.

$$\begin{aligned} \text{ArraySize} &= \text{총 Rows} \div (\text{Fetch 횟수} - 1) \\ &= 300,000 \div (401 - 1) \\ &= 300,000 \div 400 = 750 \text{ (행/Fetch)} \end{aligned}$$

→ 한 번에 750행씩 담아 Fetch는 401회뿐. 배열 인출은 이미 충분 → 왕복 가설 기각

② BCHR. 물리 대비 논리 읽기 비중을 봅니다.

$$\begin{aligned} \text{BCHR} &= ((\text{Query} + \text{Current}) - \text{Disk}) \div (\text{Query} + \text{Current}) \times 100 \\ &= (300,000 + 0 - 30,000) \div 300,000 \times 100 \\ &= 270,000 \div 300,000 \times 100 = 90.0\% \end{aligned}$$

$$\text{물리 읽기 비중} = 30,000 \div 300,000 \times 100 = 10.0\%$$

→ 논리 읽기 30만 중 디스크에서 올라온 것은 3만(10%)뿐 → "물리가 뿌리" 가설도 부분적

③ 클러스터링 팩터(CF). 인덱스 cr은 테이블 노드에 누적되므로 테이블 랜덤 블록만 떼어 냅니다.

$$\begin{aligned} \text{테이블 랜덤 블록(cr)} &= \text{TABLE 노드 cr} - \text{INDEX 노드 cr} \\ &= 300,000 - 3,000 = 297,000 \end{aligned}$$

$$\begin{aligned} \text{CF 밀도} &= \text{테이블 랜덤 블록(cr)} \div \text{ROWID 수} \\ &= 297,000 \div 300,000 = 0.99 \quad \leftarrow \text{최대값 1에 근접(최악)} \end{aligned}$$

$$\begin{aligned} \text{완전 정렬 시 밀도} &= \text{테이블 총 블록} \div \text{테이블 총 행} \\ &= 80,000 \div 80,000,000 = 0.001 \quad (\text{블록당 1,000행}) \end{aligned}$$

$$\text{악화 배수} = 0.99 \div 0.001 = 990\text{배}$$

CF 밀도 0.99는 거의 1 — 인덱스로 뽑은 행 하나하나가 저마다 다른 테이블 블록에 흩어져 있어, 300,000행을 읽으며 297,000번의 테이블 랜덤 블록 액세스가 일어났다는 뜻입니다. 완전히 정렬됐다면 블록당 1,000행씩 담겨 약 300블록이면 될 것을, 297,000블록을 방문한 셈입니다. 세 지표를 종합합니다.

ArraySize 750 → 배열 인출 정상, Fetch 401회 → 왕복 병목 아님(가설 1 기각)

BCHR 90.0% → 물리는 10%(3만)뿐, 나머지 90%는 캐시에서 논리적으로 방문 → 물리 자체가 뿌리 아님(가설 2 기각)

CF 밀도 0.99 → 인덱스로 뽑은 행마다 다른 블록 → 테이블 랜덤 액세스 297,000회 → 병목의 뿌리

시간·논리 읽기도 이 랜덤 액세스에 몰립니다.

$$\begin{aligned} \text{INDEX RANGE SCAN cr} &= 3,000 \text{ (전체 cr 300,000의 1.0\%)} && \leftarrow \text{인덱스는 가벼움} \\ \text{TABLE 랜덤 액세스 cr} &= 297,000 \text{ (99.0\%)} && \leftarrow \text{논리 읽기의 대부분} \\ \text{TABLE 랜덤 액세스 time} &= 49\text{초 (전체 50초의 약 98\%)} && \leftarrow \text{시간의 대부분} \\ \text{CPU 비중} &= 8.0 / 50.0 \times 100 = 16.0\% && \rightarrow \text{대기 바운드} \end{aligned}$$

세 지표가 함께 말해 줍니다: 배열 크기(750)는 정상이라 왕복이 아니고, BCHR 90%라 물리는 10%뿐이며, CF 밀도(0.99)가 최악이라 테이블 랜덤 액세스가 병목입니다. 물리 읽기 3만도 CF가 흠뻑린 블록을 단일 블록으로 읽다 캐시에 없던 10%일 뿐이라, 뿌리는 나쁜 CF입니다. 처방은 배열 크기 상향도 버퍼 캐시 확장도 아니라, 인덱스 키 순서로 테이블을 재정렬(CF 개선)하거나 필요한 컬럼을 인덱스에 담아 커버링으로 테이블 액세스를 없애는 것입니다. ③이 세 지표를 옹계 결합했습니다.

선택지	판정	이유
①	×	Fetch 401회에 Elapsed 50초가 잡힌 것은 맞지만 $ArraySize = 300,000 / (401 - 1) = 750$ 으로 배열 크기는 이미 큼니다. 300,000행을 401번에 나른 것은 왕복 과다가 아니라 정상이며, 시간은 왕복 대기가 아니라 테이블 노드(time 49초)에 쌓였으므로 배열 크기를 더 키워도 297,000번의 랜덤 액세스는 그대로 남습니다
②	×	테이블 노드 time 49초와 pr 30,000은 사실이나 BCHR 90.0%로 물리는 10%(3만)뿐이고, 그 물리 읽기조차 나쁜 CF가 흠뻑린 블록을 읽다 캐시에 없던 몫입니다. 버퍼 캐시를 키워 3만을 캐시 적중으로 돌려도 CF 밀도 0.99가 만든 297,000번의 논리적 테이블 랜덤 방문은 남으므로, 뿌리(CF)를 물리 읽기로 오귀속한 것입니다
③	○	ArraySize 750으로 왕복 가설을, BCHR 90%로 물리 가설을 각각 기각하고, CF 밀도 $297,000 / 300,000 = 0.99$ (최댓값 1에 근접)로 테이블 랜덤 액세스를 병목으로 지목한 뒤 CF 개선·커버링을 처방한 것이 정확합니다
④	×	테이블 노드 cr 300,000이 Rows 300,000과 1:1인 것은 맞지만, 그 99.0%(297,000)는 인덱스 스캔이 아니라 테이블 랜덤 액세스 몫입니다. INDEX RANGE SCAN cr은 3,000(1.0%)·time 0.8초뿐이라 리빌드로 리프를 다져 그 3,000을 0에 가깝게 만들어도 297,000번의 테이블 블록 방문은 그대로 남아, 원인을 인덱스로 오귀속한 것입니다

■ 이 문제의 핵심

- 세 지표로 두 가설을 각각 기각한 뒤 병목을 종합합니다. ArraySize 750은 왕복 가설을, BCHR 90.0%는 물리 가설을 기각하고, CF 밀도 0.99가 진짜 병목을 지목합니다.
- $ArraySize = 총 Rows \div (Fetch 횟수 - 1)$. $300,000 / (401 - 1) = 750$ 으로 배열 크기가 이미 커 왕복(401회)은 병목이 아닙니다 — "Fetch가 많다"는 인상만으로 왕복을 탓하면 안 됩니다.
- $BCHR = ((Query + Current) - Disk) / (Query + Current) \times 100$. $(300,000 - 30,000) / 300,000 = 90.0\%$ 로 물리는 10%(3만)뿐이라, 물리 읽기는 뿌리가 아니라 나쁜 CF의 부산물입니다.
- CF 밀도 = 테이블 랜덤 블록(cr) ÷ ROWID 수. 테이블 cr은 인덱스 cr을 뺀 297,000이고, $297,000 / 300,000 = 0.99$ 로 최댓값 1에 근접합니다(완전 정렬 시 $80,000 / 80,000,000 = 0.001$ 의 990배). 시간(49초)·논리 읽기(99.0%)가 이 노드에 몰립니다.
- 처방은 인덱스 키 순서 재정렬(CF 개선) 또는 커버링 인덱스입니다. 배열 크기·버퍼 캐시·인덱스 재생성은 297,000번의 랜덤 블록 방문을 없애지 못합니다.

■ 한 줄 정리 ArraySize 750으로 왕복 가설을, BCHR 90.0%로 물리 가설을 기각하고 CF 밀도 0.99($297,000 / 300,000$, 최댓값 1에 근접)로 테이블 랜덤 액세스를 병목으로 지목해야 하며, 배열 크기·버퍼 캐시·인덱스 재생성은 병목을 잘못 짚은 처방이다.

문제 8. 정답 ①

두 방식이 리프를 '어떤 순서로, 어떤 단위로' 읽는지가 갈림길입니다. 이름이 비슷해 반대로 볼기 쉽습니다.

Index Full Scan : 리프를 '논리적 연결 순서'로 한 블록씩 Single Block I/O
 → 인덱스 키 순서로 정렬 → ORDER BY 대체 가능
 → 대량에는 상대적으로 느림, 병렬에 부적합

Index Fast Full Scan : 세그먼트를 '물리적 저장 순서'로 Multiblock I/O
 → 정렬 보장 안 됨
 → 대량 처리·병렬에 유리

인덱스 전체를 읽는 두 방식은 정반대 성질을 가집니다. Index Full Scan은 리프를 논리적 연결 순서로 한 블록씩 읽어 정렬된 결과를 내지만 대량·병렬에는 불리합니다. Index Fast Full Scan은 세그먼트를 물리적 저장 순서로 Multiblock I/O로 읽어 대량·병렬에 유리하지만 정렬을 보장하지 못합니다.

①은 이 둘을 정반대로 뒤집었습니다. Index Full Scan에 대해 "물리적 저장 순서로 Multiblock I/O로 읽어 대량·병렬에 더 잘 맞는다"고 했지만, 그것은 Index Fast Full Scan의 성질입니다. Full Scan은 리프를 논리적 연결 순서로 한 블록씩 읽어 정렬을 얻는 대신 대량 처리 속도에서는 오히려 불리합니다. 두 방식의 읽기 방식과 강점을 서로 맞바꾼 서술입니다.

②·③·④는 옳습니다. Full Scan이 리프 연결 순서로 읽어 정렬을 얻는다는 점(②), Fast Full Scan이 물리적 Multiblock으로 읽어 대량·병렬에 유리하되 정렬은 못 낸다는 점(③), 그리고 커버링 가능 여부는 스캔 방식과 별개라는 점(④)이 모두 인덱스 스캔의 실제 동작에 부합합니다.

선택지	판정	이유
①	×	물리적 저장 순서·Multiblock·대량·병렬은 Index Fast Full Scan의 성질입니다. Index Full Scan은 리프를 논리적 연결 순서로 한 블록씩 읽어 정렬을 얻는 대신 대량엔 불리합니다. 두 방식의 성질을 정반대로 맞바꾼 서술입니다
②	○	Index Full Scan은 리프를 좌우 연결 순서로 훑어 키 순서로 정렬된 결과를 내므로 ORDER BY 용 소트를 생략하는데 쓸 수 있습니다
③	○	Index Fast Full Scan은 세그먼트를 물리적 순서로 Multiblock I/O로 읽어 대량·병렬에 유리하지만, 연결 순서를 따르지 않아 정렬은 보장하지 못합니다
④	○	커버링 가능 여부는 스캔 방식과 별개라, 필요한 컬럼이 인덱스에 다 있으면 어느 방식이든 테이블을 되짚지 않고 인덱스만 읽어 결과를 냅니다

■ 이 문제의 핵심

1. Index Full Scan은 리프를 논리적 연결 순서로 한 블록씩 읽어 정렬된 결과를 냅니다(대량·병렬엔 불리).
2. Index Fast Full Scan은 세그먼트를 물리적 저장 순서로 Multiblock I/O로 읽어 대량·병렬에 유리하되 정렬은 못 냅니다.
3. 두 방식의 읽기 순서·단위·강점은 서로 반대이므로 헷갈려 맞바꾸지 않아야 합니다.
4. 인덱스 전용 스캔(커버링) 가능 여부는 스캔 방식과 별개로, 필요한 컬럼이 인덱스에 다 있으면 테이블 액세스를 생략합니다.

■ 한 줄 정리 물리적 Multiblock·대량·병렬은 Fast Full Scan의 성질인데, 이를 Index Full Scan에 갖다 붙인 ①은 두 방식의 성질을 정반대로 맞바꾼 서술입니다.

문제 9. 정답 ①

"변별력 높은 컬럼을 선두에"라는 지침이 어디까지 참인지를 갈라 봐야 합니다.

결합 인덱스 선두 컬럼 선정 = 두 조건이 함께 충족될 때만 효과

- (1) 그 컬럼이 조회에서 '등가(=) 조건'으로 자주 쓰인다
- (2) 그 컬럼의 변별력(카디널리티)이 높다
→ 둘 다 맞아야 스캔 시작·종료점을 좁게 잡는다

빠지면 안 되는 전제

- 조건절에 등장하지 않는 컬럼을 앞에 두면 → 그 조회에선 무용
- 선두 컬럼에 범위(<, >, LIKE) 조건이 걸리면 → 뒤 컬럼이 시작점을 못 좁힘

선두 컬럼 선정에서 변별력(카디널리티)은 여러 고려 요소 중 하나일 뿐입니다. 아무리 변별력이 높아도 그 컬럼이 조회의 조건절에 등장하지 않거나 등가가 아닌 범위 조건으로만 쓰이면, 앞에 두어도 스캔 시작·종료점을 좁히지 못합니다. 즉 "등가 조건으로 쓰이는 고변별 컬럼을 앞에"라는 부분 조건을 갖춘 지침입니다.

①은 이 부분 조건을 떼어 내고 전제로 부풀렸습니다. "조건의 형태나 등장 여부와 무관하게 카디널리티가 가장 높은 컬럼을 맨 앞에 두기만 하면 어느 쿼리에서든 스캔 범위가 가장 좁아진다"고 단정했지만, 조건절에 없는 컬럼이나 범위 조건 컬럼을 선두에 두면 오히려 범위를 좁히지 못합니다. 한정된 상황에서만 성립하는 규칙을 모든 경우로 일반화한 서술입니다.

②·③·④는 옳습니다. 컬럼 순서는 조회 패턴을 따르고 조건절에 없는 컬럼은 앞에 뒤도 무용하다는 점(②), 클러스터형 인덱스는 물리 순서를 정해 테이블당 하나만 두므로 이득 큰 경로에 배치해야 한다는 점(③), 컬럼 추가·순서가 부르는 유지 부하와 조회 이득 사이에 손익분기점이 있다는 점(④)이 모두 인덱스 설계의 실제 판단에 부합합니다.

선택지	판정	이유
①	×	선두 컬럼은 '등가 조건으로 쓰이는 고변별 컬럼'이라야 범위를 좁힙니다. 조건 형태·등장 여부와 무관하게 최고 카디널리티 컬럼만 앞에 두면 된다는 것은 한정된 지침을 전제로 부풀린 서술입니다
②	○	컬럼 순서는 조회 패턴을 따라야 하고, 선두는 등가 조건으로 자주 쓰이는 컬럼이라야 시작·종료점을 좁힙니다. 조건절에 없는 컬럼은 앞에 뒤도 무용합니다
③	○	클러스터형 인덱스(IOT)는 리프가 곧 데이터라 물리 저장 순서를 정하므로 테이블당 하나만 둘 수 있어, 이득이 큰 액세스 경로에 신중히 배치해야 합니다
④	○	컬럼 추가·순서 변경은 인덱스 크기와 DML 유지 부하를 늘리므로, 조회 이득과 유지 비용을 견주는 손익분기점 판단이 필요합니다

■ 이 문제의 핵심

1. 결합 인덱스 선두 컬럼은 조회에서 등가(=) 조건으로 자주 쓰이면서 변별력이 높은 컬럼이라야 스캔 범위를 좁힙니다.
2. 변별력(카디널리티)은 여러 조건 중 하나일 뿐, 조건절에 없거나 범위 조건인 컬럼을 앞에 두면 범위를 못 좁힙니다.

- 클러스터형 인덱스(IOT)는 리프가 곧 데이터라 물리 순서를 정하므로 테이블당 하나만 두어, 이득 큰 경로에 배정합니다.
- 인덱스 설계는 조회 이득과 DML 유지 부하의 손익분기점을 함께 따져야 합니다.
 - 한 줄 정리 "등가 조건으로 쓰이는 고변별 컬럼을 선두에"라는 한정된 지침을 조건 형태와 무관하게 최고 카디널리티 컬럼만 앞에 두면 된다고 부풀린 ①이, 부분 규칙을 전체로 일반화한 부적절한 서술입니다.

문제 10. 정답 ②

3-way 플랜은 가장 깊이 왼쪽에 있는 테이블이 선행(드라이빙) 이고, 위에서 아래로 NESTED LOOPS가 겹칠수록 조인 순서가 이어집니다. 두 좌표 — 조인 순서, 조인 방식 — 로 읽습니다.

Id 3 TABLE ACCESS FULL 위판장 ← 안쪽 NL(Id 2)의 첫 자식 = 선행(드라이빙)
 Id 4 ...BY INDEX ROWID 위판낙찰 ← 안쪽 NL의 후행 = 두 번째
 Id 6 ...BY INDEX ROWID 낙찰명세 ← 바깥 NL(Id 1)의 후행 = 세 번째

조인 순서: 위판장 → 위판낙찰 → 낙찰명세. 안쪽 NESTED LOOPS(Id 2)가 위판장 ☒ 위판낙찰을, 바깥 NESTED LOOPS(Id 1)가 그 결과에 낙찰명세를 잇습니다. 선행은 가장 깊은 왼쪽인 위판장(Id 3)입니다.

조인 방식: NESTED LOOPS 두 개뿐이고 HASH JOIN·MERGE JOIN 노드가 없으므로 두 조인 모두 NL이며, 후행 위판낙찰·낙찰명세는 인덱스(Id 5·Id 7)로 탐색됩니다.

이 순서(LEADING(g e d))와 방식(둘 다 NL)을 만드는 힌트는 세 가지 표기가 모두 같은 플랜을 냅니다.

- ① LEADING(g e d) USE_NL(e) USE_NL(d) : 선행 위판장, 둘 다 NL → Id 1~7과 일치
- ③ ORDERED USE_NL(e) USE_NL(d) : ORDERED=FROM 순서(g,e,d)를 조인 순서로 → LEADING(g e d)와 동일
- ④ LEADING(g e d) USE_NL(e d) : USE_NL(e d)=위판낙찰·낙찰명세 모두 NL → ①과 동일
- ② LEADING(e g d) USE_NL(g) USE_NL(d) : 선행을 위판낙찰로 지정 → 플랜은 위판장이 선행

ORDERED는 FROM 절 순서(위판장 g, 위판낙찰 e, 낙찰명세 d)를 그대로 조인 순서로 삼으므로 LEADING(g e d)와 같고, USE_NL(e d)는 USE_NL(e) USE_NL(d)와 같은 지정입니다. 그래서 ①·③·④는 선행=위판장 + 두 조인 NL이라는 동일한 플랜을 냅니다.

②만 LEADING(e g d)로 위판낙찰을 선행으로 올립니다. 위판낙찰은 위판장코드로 위판장과, 낙찰번호로 낙찰명세와 모두 이어지므로 이 순서 자체는 유효한 조인 순서이지만, 그 플랜은 가장 깊은 왼쪽에 위판낙찰가 오게 되어 위판장(Id 3)이 선행인 위 플랜과 어긋납니다. 선행 테이블을 뒤바꾼 ②가 이 플랜을 내지 못하는 SQL입니다.

선택지	판정	이유
①	○	LEADING(g e d)로 선행=위판장(Id 3)·순서 위판장→위판낙찰→낙찰명세, USE_NL 둘로 두 조인 모두 NL — Id 1~7의 겹친 NESTED LOOPS 구조를 그대로 냅니다
②	×	LEADING(e g d)가 위판낙찰을 선행으로 지정해 가장 깊은 왼쪽이 위판낙찰가 됩니다. 플랜의 최심 왼쪽(Id 3)은 위판장이므로 선행을 뒤바꾼 SQL이고, 이 플랜을 내지 못합니다
③	○	ORDERED는 FROM 순서(위판장 g, 위판낙찰 e, 낙찰명세 d)를 조인 순서로 삼아 LEADING(g e d)와 동일하고, USE_NL 둘로 두 조인이 NL이라 ①과 같은 플랜을 냅니다
④	○	USE_NL(e d)는 위판낙찰·낙찰명세를 함께 NL로 지정한 것이라 USE_NL(e) USE_NL(d)와 같고, LEADING(g e d)까지 동일해 ①과 같은 플랜을 냅니다

■ 이 문제의 핵심

- 3-way 플랜은 가장 깊은 왼쪽 테이블이 선행입니다. 겹친 NESTED LOOPS에서 안쪽 NL(Id 2)이 위판장 ☒ 위판낙찰을, 바깥 NL(Id 1)이 낙찰명세를 잇습니다 — 순서는 위판장→위판낙찰→낙찰명세.
- NESTED LOOPS만 있고 HASH/MERGE 노드가 없으면 두 조인 모두 NL이며, 후행은 인덱스(Id 5·Id 7)로 탐색됩니다.
- 같은 조인 순서·방식을 여러 힌트 표기가 낸다. ORDERED(FROM 순서)·USE_NL(e d)(다중 지정)는 각각 LEADING(g e d)·USE_NL(e) USE_NL(d)와 같아 ①·③·④가 동일 플랜을 냅니다.
- 선행을 바꾸면 플랜이 바뀝니다. ②의 LEADING(e g d)는 위판낙찰을 선행으로 올려 최심 왼쪽이 달라지므로 이 플랜을 못 냅니다 — 선행 테이블을 뒤바꾼 값스왑입니다.

■ 한 줄 정리 플랜이 선행=위판장(Id 3)·두 조인 모두 NL이므로 LEADING(g e d)+NL을 뜻하는 ①·③·④는 같은 플랜을 내지만, 위판낙찰을 선행으로 뒤바꾼 LEADING(e g d)의 ②만 이 플랜을 내지 못한다.

문제 11. 정답 ②

세미 조인과 안티 조인은 '짝(매칭)이 있느냐 없느냐'로 갈리는 한 쌍입니다.

세미(semi) 조인 : '짝이 하나라도 있으면' 왼쪽 행을 통과 → EXISTS / IN 이 변환됨
 안티(anti) 조인 : '짝이 하나도 없어야' 왼쪽 행을 통과 → NOT EXISTS / NOT IN 이 변환됨

MINUS(차집합) : 앞 집합에서 뒤 집합에 있는 행을 빼고 남김
 → '뒤 집합에 짝이 없는 행만 남긴다'는 점에서 안티 조인과 성격이 통합

②는 이 대응을 정확히 짚었습니다. **EXISTS·IN**은 매칭이 하나라도 있으면 통과시키는 세미 조인으로, **NOT EXISTS·NOT IN**은 매칭이 없어야 통과시키는 안티 조인으로 풀립니다. **MINUS**는 앞 집합에서 뒤 집합에 없는 행만 남기므로, '짝 없는 행만 통과'라는 안티 조인의 성격과 통합입니다. 그래서 옳은 것은 ②입니다.

나머지는 모두 키워드에서 결론으로 곧장 건너뛴 반사적 연상입니다.

①: 'IN=세미 조인'이라는 반사입니다. **IN**이 세미 조인의 대표 사례인 것까지는 맞지만, 서브쿼리 쪽 컬럼이 유일키라 한 행에 여러 건이 매칭될 수 없으면 왼쪽 행이 증식할 위험이 없어 옵티마이저는 이를 일반 조인으로 풀 수 있습니다. 키워드가 변환을 결정하는 것이 아니라 중복 매칭 가능성이 결정합니다.

③: 'MINUS=안티 조인이니 똑같다'는 반사입니다. **MINUS**가 뒤 집합에 짝 없는 행을 남기는 것은 맞지만, 거기에 더해 양쪽을 정렬·해시로 중복 제거까지 하는 연산이라 결과 집합부터가 **NOT EXISTS** 안티와 다릅니다. 안티 조인과 완전히 같지 않고 정렬·해시 단계가 사라지지도 않습니다.

④: '비등치=해시 불가=조인 불가'라는 반사입니다. 등치가 아니어서 **HASH JOIN SEMI/ANTI**가 안 되는 것까지는 맞지만, 세미·안티는 해시만 전제하는 변환이 아니어서 비등치 조건은 **NESTED LOOPS SEMI/ANTI**(또는 소트 머지 계열)로 풀립니다. 변환 자체를 포기해 단순 필터로 떨어뜨리는 것이 아닙니다.

선택지	판정	이유
①	×	IN 이 세미 조인의 대표 사례인 것은 맞지만, 서브쿼리 쪽이 유일키라 중복 매칭이 없으면 왼쪽 행 증식 위험이 없어 일반 조인으로 풀 수 있습니다. 키워드가 변환을 결정한다며 일반 조인 경우를 지운 반사적 연상입니다
②	○	EXISTS·IN →세미, NOT EXISTS·NOT IN →안티로 풀리고, MINUS 도 뒤 집합에 짝 없는 행만 남긴다는 점에서 안티 조인과 성격이 통합입니다
③	×	뒤 집합에 짝 없는 행을 남긴다는 성격은 맞지만, MINUS 는 거기에 더해 양쪽 중복 제거(정렬·해시)까지 하는 연산이라 NOT EXISTS 안티와 결과 집합부터 다릅니다. 'MINUS=안티 조인'이라는 반사로 정렬·해시 단계를 지운 진술입니다
④	×	비등치라 HASH JOIN SEMI/ANTI 가 안 되는 것은 맞지만 세미·안티가 해시 매칭을 전제하는 변환은 아니어서, NESTED LOOPS SEMI/ANTI 로 풀립니다. '비등치=해시 불가=변환 포기'로 건너뛴 반사적 연상입니다

■ 이 문제의 핵심

1. 세미 조인은 '짝 있으면 통과', 안티 조인은 '짝 없어야 통과' 하는 한 쌍입니다. **EXISTS·IN**이 세미로, **NOT EXISTS·NOT IN**이 안티로 풀립니다.
2. **MINUS**는 안티 조인과 성격이 통한다. 앞 집합에서 뒤 집합에 짝이 없는 행만 남기지만, 양쪽 중복 제거(정렬·해시)까지 하므로 완전히 같지는 않습니다.
3. **IN**이라고 무조건 세미 조인은 아니다. 서브쿼리 쪽이 유일하면 중복 증식이 없어 일반 조인으로 풀 수 있습니다.
4. 비등치도 세미·안티로 풀린다. 해시가 안 될 뿐 **NESTED LOOPS SEMI/ANTI**로 처리되며, 변환을 포기하는 것이 아닙니다.

■ 한 줄 정리 **EXISTS·IN**은 세미, **NOT EXISTS·NOT IN**은 안티로 풀리고 **MINUS**도 짝 없는 행만 남긴다는 점에서 안티와 통하는 ②가 옳으며, 'IN=세미'·'MINUS=안티'·'비등치=조인 불가'는 키워드에서 곧장 건너뛴 반사적 연상입니다.

문제 12. 정답 ④

MERGE 힌트는 병합이 비용상 유리한지에 대한 옵티마이저의 판단을 눌러 병합을 유도하는 요청일 뿐, 병합해도 되는지에 대한 의미(정합성) 판단까지 뒤집는 명령이 아닙니다.

병합 여부를 가르는 두 층

1층 (의미) : 병합해도 결과가 같은가? ← **ROWNUM·UNION·분석함수** 등이 있으면 '불가'로 못박힘

2층 (비용) : 병합이 더 싸가? ← **MERGE / NO_MERGE** 힌트가 개입하는 층

⇒ 힌트는 2층에서만 작동한다. 1층에서 '불가'로 걸린 뷰는 힌트로도 병합되지 않는다.

④는 "병합할 수 없다고 판단한 뷰라도 **MERGE** 힌트면 합쳐진다"고 했습니다. 그러나 **ROWNUM·UNION·분석함수**처럼 병합하면 결과 의미가 달라지는 뷰는 의미상 병합이 금지되므로, **MERGE** 힌트를 붙여도 옵티마이저는 이를 무시하고 뷰를 독립 블록으로 남깁니다. '**MERGE** 힌트=무조건 병합'이라는 반사적 연상이라 ④가 부적절합니다.

①②③은 모두 옳습니다.

①: 단순 뷰 병합은 정의를 그대로 풀어 넣어 한 블록으로 합쳐, 조인 순서·액세스 경로 탐색의 폭을 넓힙니다.

②: 복합 뷰도 조건이 맞으면 병합되며, 집계·중복 제거의 수행 시점을 함께 재배치합니다.

③: **ROWNUM·UNION·분석함수**가 든 뷰는 병합 시 의미가 달라져 병합되지 못하고, 독립 블록으로 수행된 뒤 결합됩니다.

선택지	판정	이유
①	○	단순 뷰 병합은 정의를 바깥 블록에 그대로 풀어 넣어 뷰 경계를 없애고, 조인 순서·액세스 경로 탐색의 폭을 넓힙니다
②	○	GROUP BY·DISTINCT 복합 뷰도 조건이 맞으면 병합되며, 집계·중복 제거의 수행 시점을 함께 옮겨 재구성합니다
③	○	ROWNUM·UNION ·분석함수처럼 병합하면 의미가 달라지는 요소가 있는 뷰는 병합되지 못하고 독립 블록으로 수행됩니다
④	×	의미상 병합이 금지된 뷰(ROWNUM·UNION 등)는 MERGE 힌트로도 병합되지 않습니다. 힌트는 비용 판단만 누를 뿐이라, '힌트=무조건 병합'은 반사적 연상입니다

▪ 이 문제의 핵심

1. 단순 뷰 병합은 정의를 바깥에 풀어 넣기. 뷰 경계가 사라져 조인 순서·경로 탐색 폭이 넓어집니다.
2. 복합 뷰도 병합될 수 있다. **GROUP BY·DISTINCT**가 있어도 조건이 맞으면 집계 시점을 옮겨 재구성합니다.
3. 의미상 병합 불가능한 뷰가 있다. **ROWNUM·UNION**·분석함수가 든 뷰는 병합하면 결과가 달라져 독립 블록으로 남습니다.
4. 힌트는 비용 판단만 누른다. **MERGE** 힌트는 병합을 유도할 뿐, 의미상 금지된 병합을 성사시키지는 못합니다 — '힌트=무조건 병합'은 반사적 연상입니다.

▪ 한 줄 정리 **MERGE** 힌트는 비용 판단만 누르는 요청이라 **ROWNUM·UNION**처럼 의미상 병합이 금지된 뷰는 힌트로도 합쳐지지 않는데, "힌트만 붙이면 병합된다"는 ④는 '**MERGE** 힌트=무조건 병합' 반사에 기댄 진술이라 부적절합니다.

문제 13. 정답 ②

커서 공유의 1차 열쇠는 SQL 텍스트의 일치입니다. 오라클은 발행된 SQL 문자열을 해시해 라이브러리 캐시에서 같은 텍스트의 부모 커서를 찾고, 없으면 하드파싱해 새 커서를 만듭니다. 기본값 **CURSOR_SHARING=EXACT**에서는 텍스트를 있는 그대로 비교할 뿐, WHERE 절의 리터럴 상수를 알아서 바인드로 바꿔 정규화해 주지 않습니다. 따라서 등록번호 값이 다른 두 리터럴 SELECT는 서로 다른 텍스트 → 서로 다른 커서 → 각각 하드파싱입니다.

②는 "리터럴이라도 오라클이 상수를 자동 정규화해 하나의 커서로 묶는다"고 했는데, 이는 정규화 가정의 오류입니다. 상수를 바인드로 치환해 텍스트를 통일하는 일은 **CURSOR_SHARING=FORCE**(또는 **SIMILAR**)를 켜둘 때 비로소 일어나며, 기본값 **EXACT**에서는 값이 다르면 텍스트가 달라 공유되지 않습니다. 그래서 부적절한 진술은 ②입니다.

①③④는 각각 바인드 방식의 텍스트 동일 → 소프트파싱(①), **FORCE** 설정의 상수→시스템 바인드 치환으로 커서 수렴(③), 대소문자·공백·주석 차이만으로도 별도 커서·하드파싱(④)을 옮겨 서술합니다.

선택지	판정	이유
①	○	바인드 방식은 :b1 값과 무관하게 텍스트가 하나라, 만들어 둔 커서를 찾아 재사용하는 소프트파싱으로 처리됩니다
②	×	기본값 EXACT 는 텍스트를 그대로 비교하며 리터럴 상수를 자동 정규화하지 않습니다. 값이 다른 두 리터럴 SELECT는 텍스트가 달라 각각 하드파싱되므로 "하나의 커서로 공유"는 성립하지 않습니다
③	○	CURSOR_SHARING=FORCE 는 리터럴을 시스템 바인드로 치환한 뒤 캐시를 조회하므로 값만 다른 SELECT가 하나의 공유 커서로 수렴합니다
④	○	텍스트가 대소문자·공백·주석만 달라도 다른 SQL로 취급되어 별도 커서가 만들어지고, 리터럴이 박힌 두 SELECT는 서로를 캐시에서 찾지 못해 각각 하드파싱됩니다

▪ 이 문제의 핵심

1. 커서 공유의 1차 기준은 SQL 텍스트 일치입니다. 라이브러리 캐시는 발행 문자열을 해시해 같은 텍스트의 커서를 찾고, 없으면 하드파싱합니다.
2. 기본값 **EXACT**는 리터럴을 정규화하지 않습니다. 값이 다른 리터럴 SELECT는 텍스트가 달라 서로 다른 커서로 갈려 각각 하드파싱됩니다.
3. 바인드 변수는 텍스트를 하나로 고정합니다. 값이 달라도 커서가 하나라 소프트파싱으로 재사용되어 파싱 부하가 줄어듭니다.
4. 상수→바인드 치환은 **CURSOR_SHARING=FORCE**의 몫입니다. 텍스트 통일이 필요하면 리터럴을 시스템 바인드로 치환하도록 설정을 바꿔야 하며, 기본값에서 저절로 되지 않습니다.

▪ 한 줄 정리 기본값 **EXACT**에서는 리터럴 상수가 자동 정규화되지 않아 값이 다른 SELECT마다 텍스트가 갈려 각각 하드파싱되므로, "리터럴이라도 자동 정규화로 한 커서를 공유해 하드파싱이 없다"는 ②는 정규화 가정의 오류라

부적절합니다.

문제 14. 정답 ④

Cost가 '무엇을 비교하는 값인가'라는 측과, 그 값이 '무엇을 뜻하는가'라는 측을 섞지 않아야 합니다.

Cost의 쓰임 = 한 SQL 안에서 후보 계획들을 서로 견주기 위한 상대적 순위 지표

- 유효 : 같은 SQL의 계획 A vs 계획 B (어느 쪽이 더 싼가)
- 무효 : 다른 SQL_1의 Cost vs SQL_2의 Cost (읽는 데이터 · 연산량이 애초에 다름)

Cost 값의 절대 크기 ≠ 실제 자원량 · 경과 시간, ≠ 플랜 라인 수

④는 Cost의 유효 비교 범위를 정확히 짚습니다. Cost는 옵티마이저가 한 SQL의 여러 후보 계획 중 더 싼 쪽을 고르려고 만든 내부 순위 지표라, 같은 SQL 안에서만 우열 비교가 의미를 갖습니다. 서로 다른 SQL은 읽어야 할 데이터량과 연산 자체가 달라, Cost 숫자를 맞대어 어느 쪽이 빠른지 가릴 수 없습니다. 그래서 옳은 것은 ④입니다.

나머지는 서로 독립인 측을 인과로 엮은 별개 측 혼동입니다.

①: '한 SQL 안 계획 순위'라는 측을 'SQL 간 성능 비교'라는 다른 측으로 넘겼습니다. Cost가 I/O · CPU를 환산한 값인 것은 맞지만 그 환산은 해당 SQL이 읽어야 할 데이터를 전제로 매겨지므로, 읽는 대상이 다른 두 SQL의 Cost는 같은 척도가 아닙니다.

②: '통계 정확도' 측과 '계획 생성 가능 여부' 측을 엮었습니다. 통계가 Cost의 재료인 것은 맞지만, 통계가 없어도 옵티마이저는 동적 샘플링이나 기본값(default) 통계로 Cost를 산정해 계획을 세웁니다 — 계획의 품질이 나빠질 뿐 파싱이 실패하는 것이 아닙니다.

③: '플랜의 모양(라인 수)' 측과 'Cost' 측을 같은 것으로 봤습니다. 라인은 수행 단계를 보여 줄 뿐 단계마다 드는 비용이 제각각이라, 단계가 적어도 대량 스캔이면 Cost가 크고 단계가 많아도 인덱스로 소량만 읽으면 Cost가 작습니다. 라인 수와 Cost는 별개입니다.

선택지	판정	이유
①	×	Cost가 I/O · CPU를 환산한 값인 것은 맞지만 그 환산은 해당 SQL이 읽을 데이터를 전제로 매겨져, 읽는 대상이 다른 두 SQL의 Cost는 같은 척도가 아닙니다. 계획 순위 측을 SQL 간 성능 측으로 넘긴 혼동입니다
②	×	통계가 Cost의 재료인 것은 맞지만, 없거나 낡아도 동적 샘플링 · 기본값 통계로 Cost를 산정해 계획을 세웁니다. 계획 품질이 나빠질 뿐 파싱이 실패하지 않으니, 통계 정확도 측을 계획 생성 가능 여부 측과 엮은 오류입니다
③	×	라인은 수행 단계를 보여 줄 뿐 단계마다 드는 비용이 제각각입니다. 단계가 적어도 대량 스캔이면 Cost가 크고 단계가 많아도 소량만 읽으면 Cost가 작으므로, 플랜 라인 수와 Cost는 별개 측입니다
④	○	Cost는 한 SQL 안 후보 계획의 상대적 순위 지표라 같은 SQL의 계획 비교에는 쓰되, 서로 다른 SQL의 빠르기 비교 근거로는 삼기 어렵습니다

■ 이 문제의 핵심

1. Cost는 한 SQL 안에서만 유효한 상대 순위 지표. 같은 SQL의 후보 계획 우열을 가리는 데 씁니다.
2. SQL 간 Cost 비교는 무의미. 읽는 데이터 · 연산량이 다른 두 SQL의 Cost 숫자로 빠르기를 가릴 수 없습니다.
3. 통계가 없어도 계획은 나온다. 동적 샘플링 · 기본값 통계로 Cost를 산정하므로 파싱이 실패하지 않습니다.
4. 플랜 라인 수 ≠ Cost. 단계 수가 적다고 Cost가 낮은 것이 아니며, 무엇을 얼마나 읽느냐가 Cost를 정합니다.

■ 한 줄 정리 Cost는 한 SQL 안 후보 계획의 상대적 순위 지표라 같은 SQL 안 비교에만 유효하다는 ④가 옳고, SQL 간 Cost 비교 · 통계 없으면 파싱 실패 · 라인 수=Cost는 모두 독립인 측을 인과로 엮은 별개 측 혼동입니다.

문제 15. 정답 ③

NOLOGGING의 redo 절감은 Direct Path 적재에만 걸립니다. 세그먼트를 NOLOGGING으로 두어도, 버퍼 캐시를 거치는 일반(conventional) 인서트는 변경분을 정상적으로 redo에 기록합니다 — redo가 줄어드는 것은 APPEND(direct path)로 HWM 위 새 블록에 직접 쓰고, 그 세그먼트가 NOLOGGING이며, DB가 force logging이 아닐 때뿐입니다.

[redo 절감 성립 조건]

일반 인서트 + NOLOGGING : 버퍼 캐시 경유 → 데이터 redo 정상 기록 (절감 없음)

Direct Path(APPEND) + NOLOGGING + non-force-logging : 데이터 redo 최소화 (절감 성립)

※ NOLOGGING은 "direct path일 때만" 데이터 로깅을 건너뛴다

③은 "NOLOGGING이면 일반 인서트에도 redo 절감이 적용되어 병렬 없이 일반 INSERT만으로도 redo가 사라진다"고 했는데, 이는 실제 동작을 정반대로 뒤집은 것입니다. NOLOGGING은 conventional 경로에는 효과가 없으며, direct path가 아니면 세그먼트 속성과 무관하게 redo가 그대로 쌓입니다. 그래서 부적절한 진술은 ③입니다.

①②④는 각각 direct path의 HWM 위 적재·버퍼 캐시 우회(①), 병렬 DML의 PX별 적재와 테이블 배타 잠금(②), direct path 적재 세그먼트를 커밋 전 재접근 시 ORA-12838(④)을 옳게 서술합니다.

선택지	판정	이유
①	○	APPEND는 direct path로 HWM 위 새 블록에 기록하고 버퍼 캐시·freelist를 우회하므로 대량 적재가 빨라집니다
②	○	병렬 DML을 켜면 PX 서버가 각자 익스텐트에 direct path로 적재하고, 대상 테이블은 배타 모드로 잠겨 커밋 전까지 다른 세션의 DML이 대기합니다
③	×	NOLOGGING의 redo 절감은 direct path 경로에만 적용됩니다. 일반 인서트는 버퍼 캐시를 거쳐 데이터 redo를 정상 기록하므로 "일반 INSERT만으로 redo가 사라진다"는 정반대입니다
④	○	direct path 적재 세그먼트는 커밋 전 같은 트랜잭션에서도 재접근이 막혀, 커밋 전 SELECT 시 ORA-12838이 나머 조회하려면 먼저 커밋해야 합니다

■ 이 문제의 핵심

1. NOLOGGING의 redo 절감은 direct path 전용입니다. 일반 인서트는 세그먼트가 NOLOGGING이어도 버퍼 캐시를 거쳐 데이터 redo를 정상 기록합니다.
2. redo 절감 성립 3조건은 direct path 적재 + 세그먼트 NOLOGGING + DB non-force-logging입니다. 하나라도 빠지면 데이터 redo가 쌓입니다.
3. 병렬 DML은 테이블을 배타 잠금합니다. PX 서버가 각자 익스텐트에 적재하고 커밋 전까지 같은 테이블의 다른 DML이 대기합니다.
4. direct path 적재분은 커밋 전 재접근 불가입니다. 같은 트랜잭션이라도 커밋 전 SELECT·수정은 ORA-12838로 막힙니다.

■ 한 줄 정리 NOLOGGING은 direct path 경로에서만 데이터 redo를 줄이고 일반 인서트에는 효과가 없는데, ③은 "일반 INSERT에도 redo 절감이 적용되어 redo가 사라진다"고 정반대로 서술해 부적절합니다.

문제 16. 정답 ④

RANGE 파티션 프루닝은 조건이 파티션 키(발급일자)에 가공이 없는 범위·등치일 때, 그 값을 경계와 대조해 접근할 파티션을 골라냅니다. 이 테이블은 INTERVAL이 아니라 경계를 직접 나열한 다분할 RANGE이며, 상한을 **p_max VALUES LESS THAN (MAXVALUE)**로 열어 둔 구성입니다.

[조회·저장별 판정]

- ① TO_CHAR(발급일자, 'YYYY')='2024' → 키를 형변환·가공 → 경계 대조 불가 → 전 파티션 스캔
 ② 면허종별='제1종' → 면허종별은 파티션 키가 아닌 → RANGE 프루닝 없음
 ③ 2026년 insert 시 p_2026 자동 생성 → INTERVAL이 아니므로 자동 생성 없음 → p_max에 저장
 ④ 발급일자 >= 2024-01-01 AND < 2025-01-01 → 키에 가공 없는 범위 → p_2024만 접근, 나머지 제외 ○

④는 파티션 키인 발급일자를 가공 없이 범위로 걸었습니다. 경계값(2024-01-01 ~ 2025-01-01)과 대조되어 **p_2024**만 접근하고 나머지 파티션은 스캔에서 빠집니다. 프루닝 성립 조건을 정확히 서술했으므로 ④가 적절합니다.

나머지는 어긋납니다. ①은 겨냥한 연도가 **p_2024** 경계와 맞아떨어지는 것까지는 사실이나,

TO_CHAR(발급일자, 'YYYY')='2024'로 키를 형변환·가공한 순간 옵티마이저가 값을 경계와 대조할 수 없어 전 파티션을 스캔합니다 — "p_2024만 읽음"은 부분진실입니다. ②는 면허종별을 등치로 건 것은 맞지만 등치 여부가 아니라 파티션 키인지가 프루닝을 가릅니다. 면허종별은 파티션 키가 아닌 일반 필터라 값이 하나로 확정돼도 파티션을 줄이지 못합니다.

③은 2026년이 나열된 경계 밖이라는 지적까지는 맞지만, 이 테이블은 INTERVAL이 아니라 경계를 나열한 RANGE라 간격 규칙 자체가 없어 파티션 자동 생성이 일어나지 않고 2026년 데이터는 상한이 열린 **p_max**에 저장됩니다.

선택지	판정	이유
①	×	겨냥한 연도가 p_2024 VALUES LESS THAN (DATE '2025-01-01') 경계와 맞아떨어지는 것은 사실이나, TO_CHAR(발급일자, 'YYYY') 로 키를 형변환·가공하면 값을 경계와 대조할 수 없어 전 파티션을 스캔합니다 — "p_2024만 읽음"은 부분진실입니다
②	×	등치로 값이 하나로 확정된 것은 맞지만 프루닝을 가르는 것은 등치 여부가 아니라 파티션 키인지입니다. 면허종별은 파티션 키가 아닌 일반 필터라 p_2024·p_2025 로 좁혀지지 않고 전 파티션이 대상입니다
③	×	2026년이 나열된 경계 밖인 것은 맞지만 이 DDL은 INTERVAL이 아니라 경계를 나열한 RANGE라 간격 규칙 자체가 없어 파티션 자동 생성이 일어나지 않습니다. 2026년 데이터는 비어 있는 것이 아니라 상한이 열린 p_max 가 받습니다
④	○	발급일자에 가공 없는 범위 조건이라 경계와 대조되어 p_2024 만 접근하고 나머지 파티션은 스캔에서 제외됩니다

▪ 이 문제의 핵심

1. RANGE 프루닝은 파티션 키에 가공이 없어야 경계값과 대조되어 걸립니다. 발급일자를 가공 없는 범위로 걸면 해당 연도 파티션만 읽습니다.
 2. 키를 형변환·가공(TO_CHAR)하면 프루닝이 깨집니다. 경계 대조가 불가능해 전 파티션을 스캔하며, "연도를 겨냥"이라는 걸모습은 부분진실입니다.
 3. 파티션 키가 아닌 컬럼(면허종별)은 프루닝 대상이 아닙니다. 값을 좁혀도 파티션을 줄이지 못하는 일반 필터입니다.
 4. 경계를 나열한 RANGE는 자동 생성이 없습니다. INTERVAL과 달리 새 연도 파티션이 저절로 생기지 않고, 상한이 열린 **p_max**가 초과분을 받습니다.
- 한 줄 정리 발급일자를 가공 없는 범위로 건 ④만 **p_2024** 프루닝이 성립하며, 키 형변환(①)·비키 필터(②)·INTERVAL 오해(③)는 프루닝이나 자동 생성이 성립하지 않아 부적절합니다.

문제 17. 정답 ②

분배 방식은 어느 입력이 **PX SEND**를 거치고 그 SEND의 종류가 무엇인지로 읽습니다. 조인 입력 두 갈래를 나란히 봅니다.

Id 6 PX SEND BROADCAST :TQ10000 (Q1,00 → 전 서버 복제) 관로구간을 모든 서버에 뿌림
 Id 8 TABLE ACCESS FULL 관로구간 (약 4.8만 건) 복제 대상 - 소형
 Id 9 PX BLOCK ITERATOR (Q1,01 → SEND 없음) 누수감지이벤트를 제자리에서 읽음
 Id 10 TABLE ACCESS FULL 누수감지이벤트 (약 3.5억 건) 재분배 안 함 - 대형

소형 관로구간(Id 8) 위에는 **PX SEND BROADCAST**(Id 6)가 있어 각 병렬 서버에 통째로 복제됩니다.

대형 누수감지이벤트(Id 10) 위에는 **PX SEND**가 없습니다. Id 9 **PX BLOCK ITERATOR**로 각 서버가 자기 블록 범위만 읽어, 서버마다 들고 있는 관로구간 복제본과 로컬로 조인(Id 4 **HASH JOIN**)합니다.

한쪽에만 SEND(그것도 BROADCAST)가 있고 다른 쪽은 SEND 없이 제자리 스캔 — 이 조합이 broadcast 분배입니다.

대형×소형에서는 소형을 복제해도 늘어나는 양이 작고, 대형 3.5억 건을 재분배하는 왕복을 피하므로 유리합니다.

- ① hash-hash : 양쪽 위에 각각 PX SEND HASH → 대형 쪽 Id 9는 SEND 없는 BLOCK ITERATOR
 ② broadcast : 소형에 SEND BROADCAST, 대형은 SEND 없음 → Id 6·Id 8·Id 9·Id 10과 일치
 ③ full PWJ : 두 입력이 같은 PX PARTITION 아래, SEND 없음 → 소형 쪽에 SEND BROADCAST 있음
 ④ broadcast(역): 대형을 BROADCAST, 소형을 제자리 → SEND BROADCAST는 소형(Id 8) 쪽에 붙어 있음

SEND가 소형 관로구간 쪽에만 하나(Id 6 BROADCAST) 있고 대형 누수감지이벤트 쪽은 SEND 없이 **PX BLOCK ITERATOR**(Id 9)라는 사실이 hash-hash·full PWJ·방향을 뒤집은 broadcast를 한꺼번에 배제합니다. 따라서 ②가 옳습니다.

선택지	판정	이유
①	×	hash-hash면 양쪽 입력 위에 각각 PX SEND HASH 가 있어야 하는데, 대형 누수감지이벤트 쪽(Id 9)은 SEND 없는 PX BLOCK ITERATOR 이고 SEND는 소형 쪽 Id 6의 BROADCAST 하나뿐입니다. 분배 방식을 broadcast에서 hash-hash로 뒤바꾼 진술입니다
②	○	소형 관로구간(Id 8) 위 PX SEND BROADCAST (Id 6)로 전 서버에 복제하고, 대형 누수감지이벤트(Id 10)는 SEND 없이 PX BLOCK ITERATOR (Id 9)로 제자리 스캔해 로컬 조인하는 broadcast 분배와 일치합니다
③	×	full 파티션 와이즈면 두 입력이 같은 PX PARTITION 아래에서 SEND 없이 읽혀야 하는데, 소형 쪽에 PX SEND BROADCAST (Id 6)가 있고 두 테이블 모두 관로구간ID 기준 파티션도 아닙니다
④	×	PX SEND BROADCAST (Id 6)는 소형 관로구간(Id 8) 쪽에 붙어 있고, 대형 누수감지이벤트(Id 10)는 Id 9 PX BLOCK ITERATOR 로 제자리 스캔합니다. 복제되는 테이블을 대형·소형으로 뒤바꾼 값스왑입니다

▪ 이 문제의 핵심

1. 분배 방식은 조인 입력의 **PX SEND** 유무와 종류로 판별합니다. 여기서는 소형 쪽(Id 6)에만 **PX SEND BROADCAST**가 있고 대형 쪽(Id 9)은 SEND가 없습니다.
 2. broadcast는 대형×소형에서 소형을 전 서버에 복제하고 대형은 제자리에서 읽는 방식입니다. 각 서버가 소형 복제본과 자기 블록의 대형 행을 로컬 조인합니다.
 3. 한쪽에만 SEND(BROADCAST)가 있고 다른 쪽은 SEND 없이 **PX BLOCK ITERATOR**라는 점이 핵심 단서입니다. 양쪽에 SEND HASH가 있으면 hash-hash, SEND가 아예 없으면 full PWJ입니다.
 4. 복제 방향이 중요합니다. BROADCAST는 소형(Id 8) 쪽에 붙어야 하며, 대형(3.5억)을 복제한다고 보면 방향을 뒤바꾼 값스왑입니다 — 대형을 복제하면 서버 수만큼 붙어나 오히려 불리합니다.
- 한 줄 정리 소형 관로구간(Id 8) 위에만 **PX SEND BROADCAST**(Id 6)가 있고 대형 누수감지이벤트(Id 10)는 SEND 없이 **PX BLOCK ITERATOR**(Id 9)로 제자리 스캔해 로컬 조인하므로, 소형을 복제하는 broadcast 분배다.

문제 18. 정답 ④

인덱스 하나의 정렬 순서는 그 인덱스를 타는 한쪽 입력에만 붙습니다. 소트 머지 조인은 양쪽 입력을 각자 정렬해야 병합합니다.

소트 머지 조인 = 왼쪽 입력 정렬(SORT JOIN) + 오른쪽 입력 정렬(SORT JOIN) → 병합
 왼쪽 테이블에 조인 키 인덱스 → 왼쪽만 정렬된 순서로 공급 → 왼쪽 SORT JOIN 생략
 오른쪽 테이블은? 그 인덱스와 무관 → 오른쪽은 여전히 정렬해야 함(자기 인덱스가 따로 있어야 함)
 ⇒ 인덱스 '하나'로 지워지는 SORT JOIN은 한쪽뿐

④는 "조인 키에 인덱스가 하나라도 있으면 양쪽 **SORT JOIN**이 둘 다 사라진다"고 했습니다. 그러나 한 테이블의 인덱스는 그 테이블의 스캔에만 정렬된 순서를 줄 뿐, 상대 테이블 입력은 정렬해 주지 못합니다. 양쪽 **SORT JOIN**이 모두 없어지려면 양쪽 각각 조인 키로 정렬된 접근 경로가 있어야 합니다. 한쪽에만 성립하는 생각을 양쪽 전체로 부풀린 하위항목 전체화라 ④가 부적절합니다.

①②③은 모두 옳습니다.

①: **SORT ORDER BY**(결과 정렬)·**SORT UNIQUE**(중복 제거)·**SORT JOIN**(조인 키 정렬)은 저마다 도는 이유가 다른 소트 계열 오퍼레이션입니다.

②: **SORT JOIN**은 양쪽을 조인 키로 정렬해야 병합하므로, 한쪽이 인덱스로 정렬돼 공급되면 그쪽 **SORT JOIN**은 생략됩니다.

③: **SORT GROUP BY**는 그룹 키가 인덱스 선두 정렬 순서와 맞으면 인덱스를 훑는 것으로 대체돼 생략됩니다.

선택지	판정	이유
①	○	SORT ORDER BY · SORT UNIQUE · SORT JOIN 은 결과 정렬·중복 제거·조인 키 정렬 등 저마다 다른 이유로 도는 소트 계열 오퍼레이션입니다
②	○	소트 머지 조인은 양쪽을 조인 키로 정렬해야 병합하므로, 한쪽이 인덱스로 정렬돼 공급되면 그쪽 SORT JOIN 이 생략됩니다
③	○	SORT GROUP BY 는 그룹 키가 인덱스 선두 정렬 순서와 일치하면 인덱스를 훑는 것으로 대체돼 소트가 생략됩니다
④	×	한 테이블의 인덱스는 그 테이블 입력만 정렬해 줄 뿐 상대 입력은 정렬하지 못합니다. 한쪽 SORT JOIN 생략을 양쪽 둘 다로 부풀린 하위항목 전체화입니다

■ 이 문제의 핵심

- 소트 계열 오퍼레이션은 도는 이유가 제각각. 결과 정렬(**ORDER BY**)·중복 제거(**UNIQUE**)·조인 키 정렬(**SORT JOIN**)·그룹핑(**GROUP BY**)이 각기 다릅니다.
- 인덱스는 자기 순서와 맞는 소트만 지운다. 정렬 순서가 일치하는 하나의 소트만 생략됩니다.
- 소트 머지 조인은 양쪽을 각자 정렬한다. 한 테이블 인덱스는 그쪽 **SORT JOIN**만 지우고, 상대 입력은 여전히 정렬해야 합니다.
- 부분 성립을 전체로 넓히지 말 것. 양쪽 **SORT JOIN**이 다 없어지려면 양쪽 각각의 정렬된 접근 경로가 필요합니다.

■ 한 줄 정리 한 테이블의 인덱스는 그쪽 입력의 **SORT JOIN**만 지울 수 있어 양쪽 모두 없애려면 양쪽 각각 정렬 경로가 필요한데, "인덱스가 하나라도 있으면 양쪽 **SORT JOIN**이 둘 다 사라진다"는 ④는 한쪽 생략을 전체로 부풀린 하위항목 전체화입니다.

문제 19. 정답 ①

오라클 MVCC의 읽기 일관성은 커밋된 데이터만 보여 줍니다(Dirty Read 없음). READ COMMITTED에서 각 SELECT는 자기 문장이 시작되는 시점의 SCN 스냅샷으로 읽되, 그 스냅샷은 이미 커밋된 변경은 반영하고 아직 커밋되지 않은 변경은 Undo로 되감아 커밋 전 값으로 읽습니다.

[TX2 SELECT(t3) 값 계산]
 가 = 100 (변경 없음)
 나 : t1에 260으로 COMMIT → 커밋됨 → 260 (반영)
 다 : t2에 350으로 UPDATE(미커밋) → 커밋 안 됨 → Undo로 되감아 300 (커밋 전 값)
 SUM = 100 + 260 + 300 = 660

t3 문장 시작 시점에 '나'의 +60은 t1에 이미 커밋되어 스냅샷에 포함되지만, '다'의 +50은 TX1이 아직 커밋하지 않았으므로 TX2는 그 미커밋 변경을 볼 수 없습니다. TX2는 '다'의 변경 블록을 Undo로 되감아 커밋 전 값 300으로 읽습니다. 따라서 SUM = 100 + 260 + 300 = 660, 정답은 ①입니다.

②③④는 커밋/미커밋 값을 뒤섞은 값스왑 오답입니다. ②는 t2에서 갱신만 되고 커밋되지 않은 '다'(350)까지 반영해 더티 리드가 된 710, ③은 t1에 분명히 커밋된 '나'의 +60조차 반영하지 않아 초기 합에 머문 600, ④는 '나'는 커밋 전 200·'다'는 t2 변경값 350으로 두 값의 자리를 정확히 맞바꿔 읽은 650입니다.

선택지	판정	이유
①	○	커밋된 '나'(260)는 반영하고 미커밋인 '다'는 Undo로 되감아 300으로 읽어 100+260+300=660입니다
②	×	100+260+350. t2에서 갱신만 되고 커밋되지 않은 '다'의 350까지 반영한 더티 리드로, MVCC는 미커밋 값을 읽지 않고 Undo로 되감아 300으로 읽습니다
③	×	100+200+300. t1에 커밋된 '나'의 +60을 놓쳐 초기 합에 머문 값으로, READ COMMITTED는 문장 시작 전 커밋분을 반영하므로 '나'는 260이어야 합니다
④	×	100+200+350. 커밋된 '나'는 커밋 전 200으로, 미커밋인 '다'는 변경값 350으로 두 값의 자리를 맞바꿔 읽은 값스왑입니다

▪ 이 문제의 핵심

1. MVCC는 커밋된 데이터만 읽습니다(Dirty Read 없음). 미커밋 변경은 Undo로 되감아 커밋 전 값으로 읽습니다.
2. READ COMMITTED는 문장 시작 시점의 커밋분을 반영합니다. t3 이전에 커밋된 '나'의 +60은 스냅샷에 포함됩니다.
3. 커밋 여부가 값을 가릅니다. '나'는 커밋되어 260, '다'는 미커밋이라 300으로 갈려 SUM=660이 됩니다.
4. Undo는 CR 재구성의 재료입니다. 문장 스냅샷보다 최신인 미커밋 블록('다')을 Undo로 되감아 커밋 전 값으로 읽습니다.

▪ 한 줄 정리 MVCC는 커밋된 '나'(260)만 반영하고 미커밋인 '다'는 Undo로 되감아 300으로 읽으므로 SUM=100+260+300=660(①)이며, 더티 리드(710)·커밋 누락(600)·자리바꿈(650)은 값스왑 오답입니다.

문제 20. 정답 ③

교착(Deadlock)의 성립 요건은 대기의 순환입니다. 표에서 53은 R100을 보유한 채 R200을 기다리고, 57은 R200을 보유한 채 R100을 기다립니다 — 53→57→53으로 대기가 한 바퀴 돕니다. 어느 쪽도 상대가 커밋해 주기 전에는 필요한 잠금을 얻지 못하므로, 상대의 커밋을 서로 무한정 기다리게 됩니다.

[표 판독 - 순환 여부]

53 : R100 X GRANT (보유) + R200 X WAIT → 57이 쥔 R200 대기

57 : R200 X GRANT (보유) + R100 X WAIT → 53이 쥔 R100 대기

대기 방향 : 53 → 57 → 53 (순환) ⇒ 교착

단순 블로킹이면 : A는 B를 기다리되 B는 아무도 안 기다림 (순환 없음) → B 커밋 시 해소

SQL Server는 교착을 방지하지 않습니다. 백그라운드 락 모니터(deadlock monitor)가 대기 그래프의 순환을 주기적으로 탐지해, 롤백 비용이 더 작은 트랜잭션을 교착 희생자(deadlock victim, 오류 1205)로 골라 자동 롤백합니다. 그러면 희생자가 쥐고 있던 잠금이 풀려 상대 세션이 진행합니다. ③은 순환 대기 → 교착 → 자동 victim 선정·롤백을 정확히 서술했으므로 적절합니다.

나머지는 어긋납니다. ①은 53만 대기하는 단순 블로킹으로 봤지만, 표의 57도 **R100 X WAIT**라 함께 대기 중이므로 순환이며 "57이 커밋하면 풀린다"가 성립하지 않습니다(57은 커밋에 필요한 R100을 못 얻습니다). ②는 순환은 맞게 봤으나 "SQL Server가 교착을 스스로 해소하지 못한다"는 부분이 틀렸습니다 — 락 모니터가 자동으로 victim을 롤백합니다. ④는 "두 세션이 함께 X를 보유"한다는, 블로킹에도 교착에도 공통으로 나타나는 단서를 판별 근거로 삼아 교착을 부정했는데, 교착 여부를 가르는 것은 X 보유 여부가 아니라 대기의 순환입니다.

선택지	판정	이유
①	×	표의 57도 R100 X WAIT 로 함께 대기 중이라 순환입니다. 57은 R100을 얻어야 커밋할 수 있어 "57 커밋으로 자동 해소"는 성립하지 않습니다
②	×	순환은 맞지만 SQL Server는 락 모니터가 교착을 자동 감지해 victim을 롤백합니다 — "무한 대기·관리자 KILL 필요"는 틀립니다
③	○	53→57→53 순환 대기가 교착이며, 락 모니터가 롤백 비용이 작은 쪽을 victim(1205)으로 롤백하고 상대는 진행합니다
④	×	"함께 X를 보유"는 블로킹·교착 어디에나 나타나는 공통 단서라 판별 근거가 못 됩니다. 교착 여부는 대기의 순환으로 가려야 하며, 표는 순환을 보여 줍니다

▪ 이 문제의 핵심

1. 교착의 요건은 대기의 순환입니다. 53→57→53처럼 서로 상대가 쥔 잠금을 기다려 한 바퀴 돌면 교착입니다.

2. 단순 블로킹과 교착의 차이는 순환 유무입니다. 블로킹은 대기가 한 방향이라 블로커가 커밋하면 풀리지만, 교착은 어느 쪽도 먼저 풀리지 못합니다.

3. SQL Server는 교착을 자동 해소합니다. 락 모니터가 순환을 감지해 롤백 비용이 작은 쪽을 victim(오류 1205)으로 롤백하고 상대를 진행시킵니다.

4. "함께 X를 보유"는 공통 단서입니다. 블로킹·교착 모두에 나타나므로 그 자체로 교착을 부정하거나 확정하는 근거가 못 되며, 판별은 대기 순환으로 합니다.

- 한 줄 정리 53과 57이 서로 상대가 쥔 강의실 행을 기다리는 순환 대기라 교착이며 SQL Server의 락 모니터가 victim을 자동 롤백하므로 ③이 적절하고, "함께 X 보유"라는 공통 단서로 교착을 부정한 ④는 판별 근거의 오독입니다.